GALALA UNIVERSITY
Powered by
Arizona State University®

Poultry Farm Management System

CSE221: *Database systems*

Spring semester 2026

Project Team

Student ID	Student Name	Project Evaluation	
223101629	**Samaa Mohamed Saleh**	Report	
		Presentation	
		Code	
		Total	
223101614	**Omar Khalifa**	Report	
		Presentation	
		Code	
		Total	
224127559	**Mariam EL Zayat**	Report	
		Presentation	
		Code	
		Total	
224127007	**Ahmed Mohamed Ahmed**	Report	
		Presentation	
		Code	
		Total	
224124339	**Basmala Gamal Mohamed**	Report	
		Presentation	
		Code	
		Total	

Under supervision of Professor: Shaker El-Sappagh

2025-2026

GALALA UNIVERSITY
Powered by
Arizona State University®

Abstract

The commercial poultry farming industry operates under high-risk, low-margin conditions that demand precise environmental monitoring, optimized resource allocation, and strict biological containment protocols. Traditional paper-based tracking methods introduce significant human error, resulting in uncontrolled mortality spikes, suboptimal Feed Conversion Ratios (FCR), and untraceable financial losses.

This project delivers the Smart Poultry Farm Management System, a comprehensive, enterprise-grade relational database application with an integrated graphical user interface. The system is built on a highly normalized relational schema (Third Normal Form — 3NF) comprising 35 tables, 16 triggers, and 10 database views, all deployed on MySQL 8.0. The application layer is implemented using Node.js with Express.js, rendered through EJS templates, and secured with role-based access control (RBAC) across three distinct user roles: Manager, Farmer Worker, and Technician.

The platform manages the complete farm lifecycle: multi-user authentication, concurrent biological breeding cycles, high-frequency daily telemetry capture (temperature, humidity, ammonia, water intake, feed consumption, and mortality), a full supply chain (suppliers, purchase orders, and inventory), B2B commercial operations (customers, sales orders, invoices, and payments), equipment maintenance tracking, human resources management (attendance, payroll, performance reviews, and training), quality control inspections, and a comprehensive audit trail. All data flows through enforced referential constraints, automated stock-deduction triggers, and real-time aggregate views to guarantee absolute data integrity.

GALALA UNIVERSITY
Powered by
Arizona State University®

Introduction

In the era of precision agriculture, data-driven management has shifted from an innovative luxury to a survival requirement for large-scale poultry production. Facilities managing tens of thousands of birds in enclosed environments face a reality where a 24-hour delay in identifying a disease outbreak or a sudden shift in water-to-feed consumption ratios can result in catastrophic biological and financial collapse.

This project presents a centralized software architecture designed to structuralize the complex, multi-dimensional data flows generated during agricultural breeding operations. The core philosophy is to replace fragmented flat files and disconnected spreadsheets with a robust relational Database Management System (DBMS) that enforces referential integrity and standardizes all input parameters.

By establishing clear logical dependencies between workers, biological production timelines, resource consumption rates, inventory levels, and client invoices, the architecture empowers farm managers with real-time, actionable intelligence. Through optimized relational queries and precomputed database views, administrative users can transform raw daily metrics into comprehensive performance visualizations — enabling them to minimize feed waste, protect livestock asset values, and dynamically project net market profitability per production cycle.

The Business Problem Analysis

Before formulating the logical database schema, an in-depth domain analysis of poultry operational workflows was conducted, isolating four primary structural bottlenecks:

1 Information Asymmetry and Untracked Biological Telemetry

Traditional farms lack mechanisms to systematically record daily bird mortality alongside feed consumption, environmental readings (temperature, humidity, ammonia levels), and water intake per production cycle. When a mortality spike occurs, management cannot perform retrospective data audits to correlate causes with specific breed batches, environmental anomalies, or feed quality changes. This forces the business into a reactive rather than proactive approach toward veterinary intervention.

2 Fragmented Supply Chain and Financial Leakage

Expenditures related to feed procurement, veterinary medicines, vaccines, and equipment maintenance from diverse suppliers are frequently logged in separate journals. Similarly, wholesale sales to various B2B customers are recorded independently. Without a relational architecture to compute real-time cost-benefit calculations per cycle, management operates with skewed profit margins and cannot pinpoint whether losses originate from high supplier costs, inefficient flock conversion, or excessive overhead.

3 Database Anomalies in Flat Storage Systems

Utilizing flat storage mechanisms introduces data integrity risks. For example, deleting an inactive supplier profile from a spreadsheet inadvertently destroys the historical cost archives of all associated purchase orders. Updating a medicine price in one location may not propagate to treatment cost calculations. The proposed database addresses these vulnerabilities through strict relational decomposition, foreign key constraints, and cascading policies.

4 Absence of Role-Based Operational Boundaries

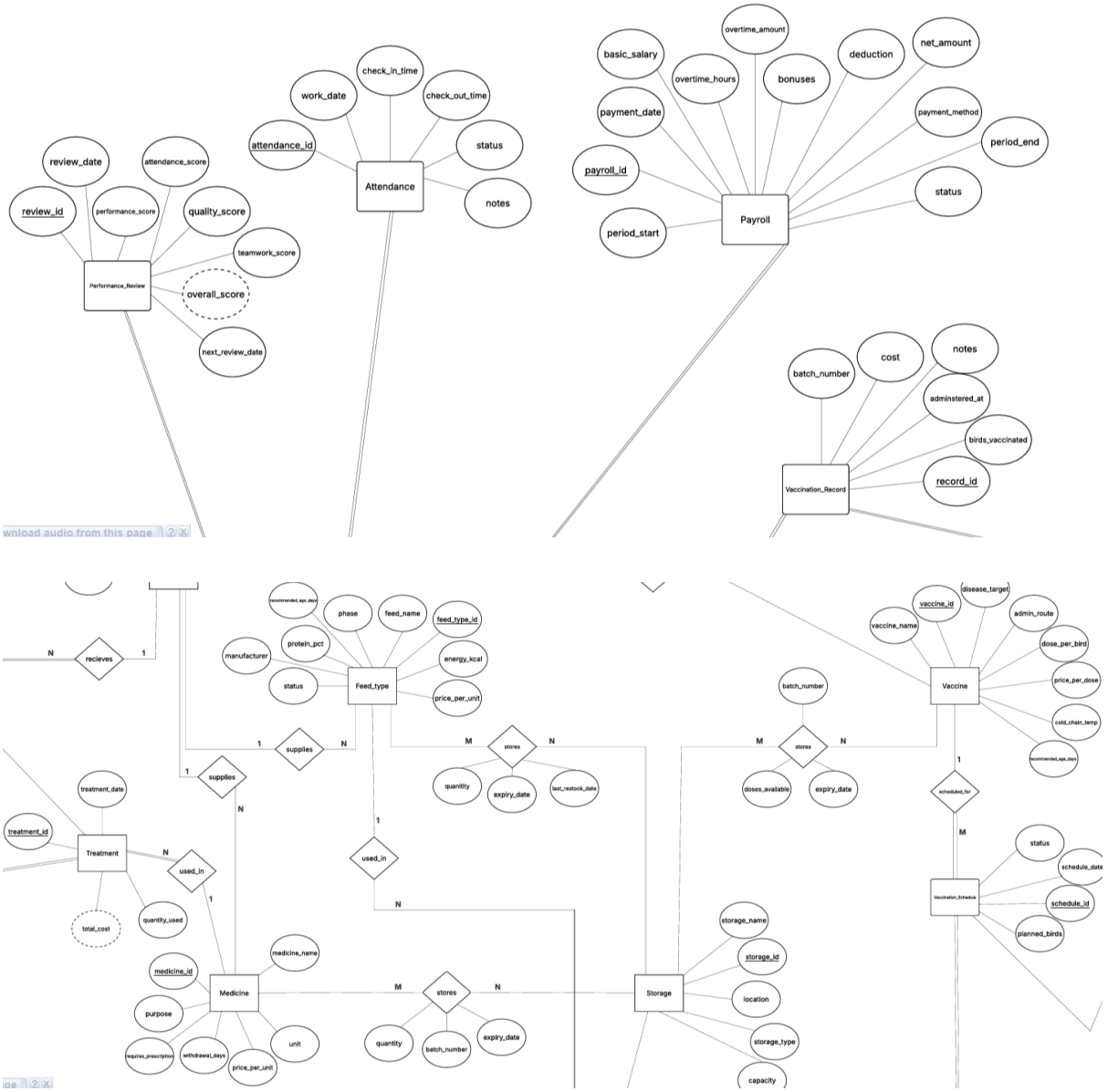
In manual systems, all personnel often have unrestricted access to all operational data, including sensitive financial records and employee salary information. This creates security vulnerabilities and accountability gaps. The system implements strict RBAC, restricting each user role to only the data and operations relevant to their responsibilities.

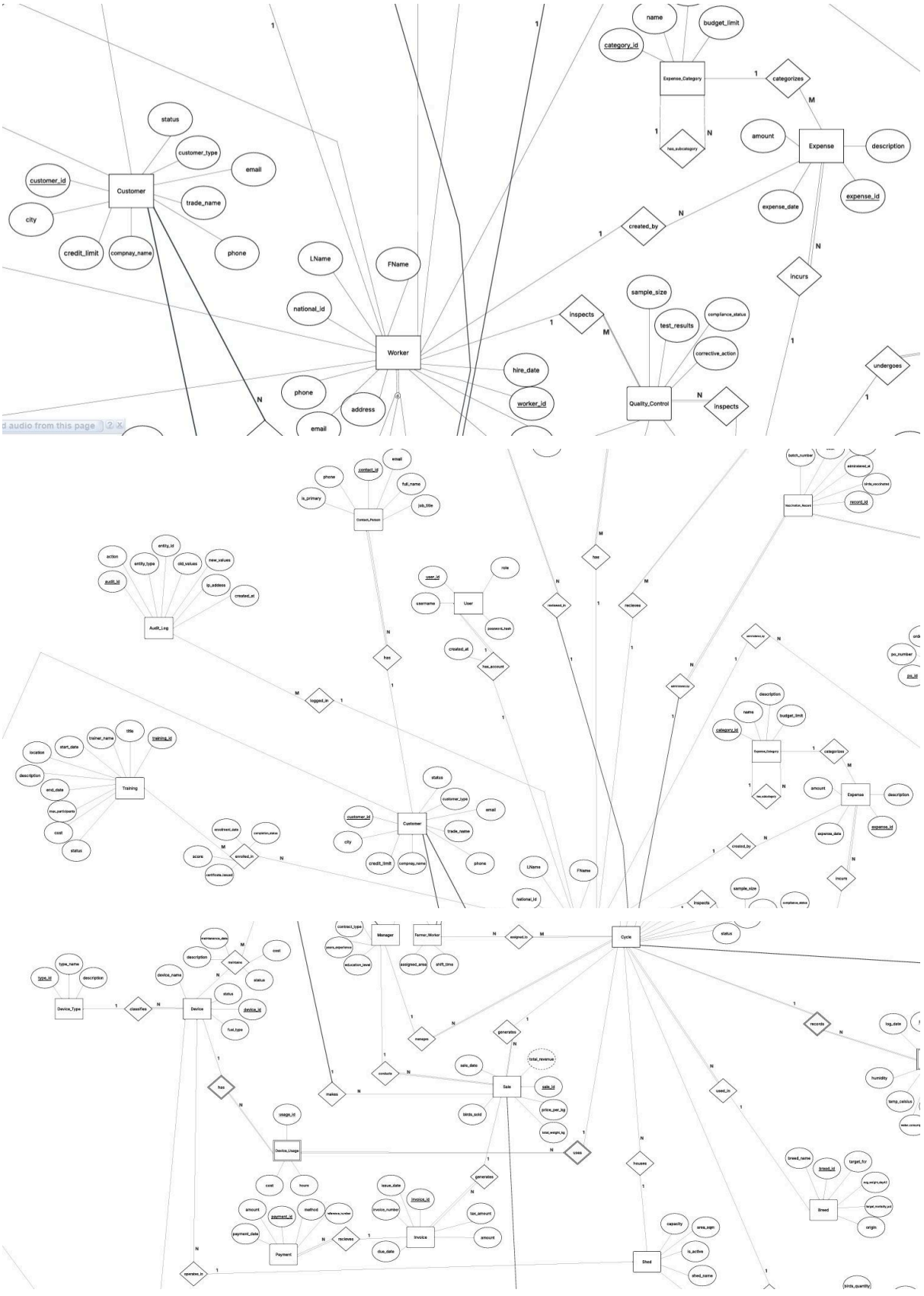
GALALA UNIVERSITY
Powered by
Arizona State University®

Database Design

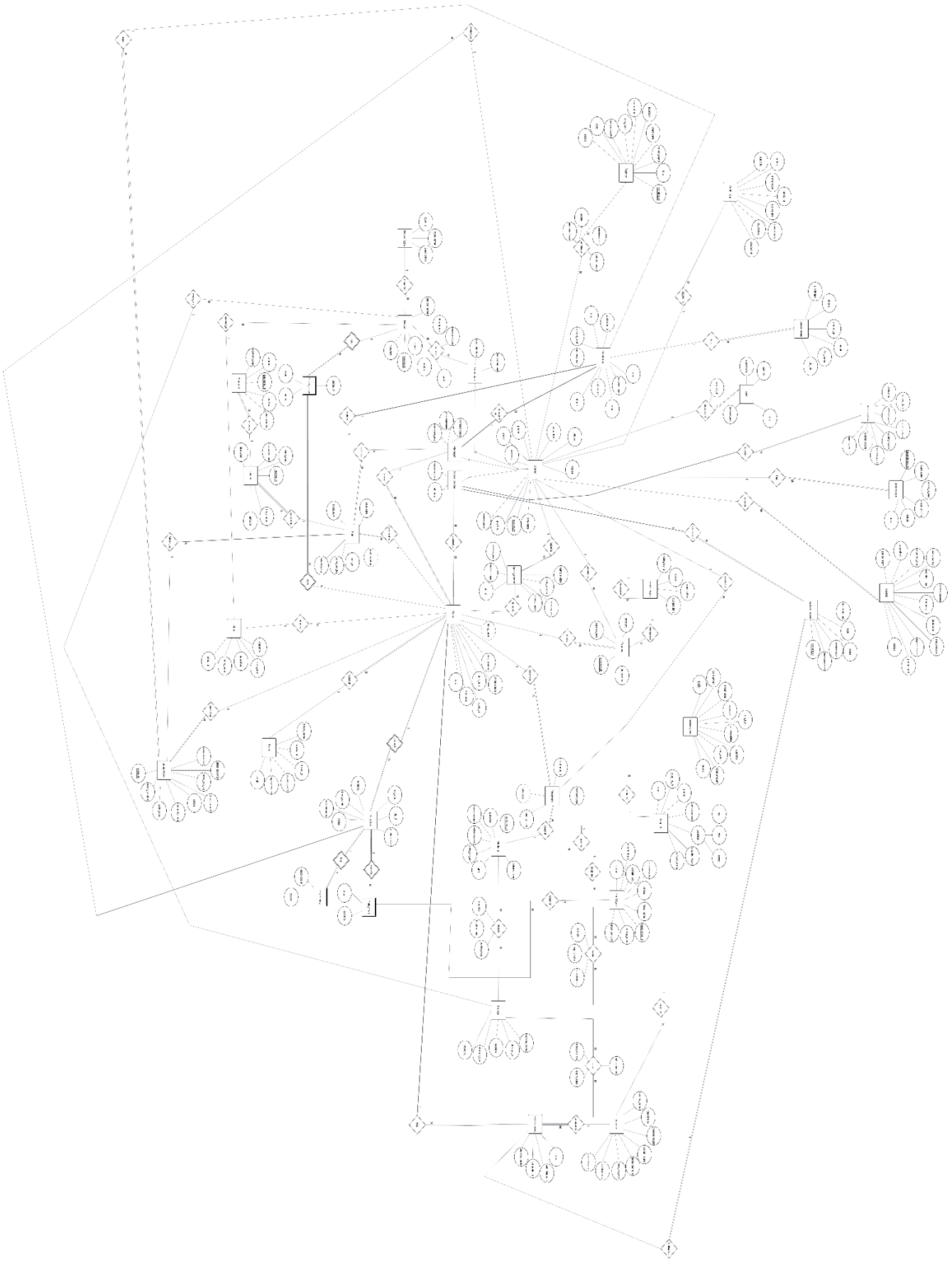
1 Entity-Relationship Model

The foundational structural blueprint was modeled using a rigorous top-down entity-relationship framework, translating intricate farming operations into a stable logical schema organized across seven functional sections.




Customer
customer_id
status
customer_type
email
trade_name
city
credit_limit
compnay_name
phone
Worker
LName
FName
national_id
phone
address
email
hire_date
worker_id
Expense_Category
name
budget_limit
category_id
has_subcategory
categorizes
Expense
amount
description
expense_id
expense_date
created_by
incurs
Quality_Control
inspects
sample_size
test_results
compliance_status
corrective_action
undergoes
Training
Device
Device_Type
Manager
Cycle
Sale
Payment
Invoice
Shed
Breed

GALALA UNIVERSITY
Powered by
Arizona State University®



2 Key Structural Design Patterns

A. Table Inheritance (IS-A / Subtyping)

The schema implements a clean IS-A inheritance hierarchy for the staffing domain. The base Worker table stores shared personal attributes (name, national ID, phone, email, salary, hire date), while three specialized subtables, Manager, Farmer_Worker, and Technician, store role-specific attributes. Each subtable uses person_id as both its Primary Key and a Foreign Key referencing Worker(person_id), establishing a 1:1 identifying relationship with ON DELETE CASCADE.

B. Weak Entities

Three entities are modeled as weak entities that cannot exist without their owner:

- **Daily_Log**: owned by Cycle; composite PK (log_id, cycle_id)
- **Daily_Feed**: owned by Daily_Log; composite PK (log_id, cycle_id, feed_type_id)
- **Daily_Mortality**: owned by Daily_Log; composite PK (log_id, cycle_id)
- **Device_Usage**: owned by Device + Cycle; composite PK (usage_id, device_id, cycle_id)

C. Cardinality Examples

Relationship	Type	Explanation
`Breed` → `Cycle`	1:N	One breed is used across many cycles; each cycle uses exactly one breed
`Shed` → `Cycle`	1:N	One shed hosts many cycles over time; each cycle occupies one shed
`Cycle` → `Daily_Log`	1:N	One cycle generates many daily logs; each log belongs to one cycle
`Supplier` → `Purchase_Order`	1:N	One supplier receives many POs; each PO targets one supplier
`Customer` → `Sales_Order`	1:N	One customer places many orders; each order belongs to one customer
`Cycle` ↔ `Farmer_Worker`	M:N	Resolved via `Assigned_To` junction table
`Device` ↔ `Cycle`	M:N	Resolved via `Device_Usage` junction table
`Training` ↔ `Worker`	M:N	Resolved via `Training_Enrollment` junction table
`Vaccination_Schedule` → `Vaccination_Record`	1:1	Each schedule has at most one completion record

3 Complete Entity Catalog

The following table documents all 35 entities with their attributes, primary keys, and foreign keys:

Section 1: Internal Staffing & User Hierarchy

Entity	Attributes	PK	FK
Worker	person_id, FName, LName, national_ID, phone, email, address, job_title, hire_date, monthly_salary, worker_type, created_at, updated_at	person_id	—
Manager	person_id, contract_type, years_experience, education_level, created_at	person_id	person_id → Worker
Farmer_Worker	person_id, assigned_area, shift_time, contract_type, created_at	person_id	person_id → Worker
Technician	person_id, specialization, license_number, created_at	person_id	person_id → Worker
User	user_id, username, password_hash, user_role, is_admin, person_id, is_active, created_at, last_login	user_id	person_id → Worker

Section 2: Supply Chain & Inventory

Entity	Attributes	PK	FK
Supplier	supplier_id, company_name, contact_person, phone, email, city, supplier_type, status, created_at	supplier_id	—
Storage	storage_id, storage_name, storage_type, capacity, location, is_active, created_at	storage_id	—
Purchase_Order	po_id, po_number, supplier_id, item_type, item_name, item_description, order_date, expected_delivery_date, delivery_date, status, quantity, unit_price, total_amount (VIRTUAL), storage_id, created_at	po_id	supplier_id → Supplier, storage_id → Storage
Feed_Type	feed_type_id, name, protein_pct, energy_kcal, manufacturer, recommended_age_days, status, supplier_id, created_at	feed_type_id	supplier_id → Supplier
Feed_Stock	storage_id, feed_type_id, quantity, reorder_level, expiry_date, last_restock_date	(storage_id, feed_type_id)	storage_id → Storage, feed_type_id → Feed_Type
Medicine	medicine_id, medicine_name, unit, price_per_unit, manufacturer, purpose, requires_prescription, withdrawal_days, supplier_id, status, created_at	medicine_id	supplier_id → Supplier
Medicine_Stock	storage_id, medicine_id, quantity, reorder_level, expiry_date, batch_number	(storage_id, medicine_id)	storage_id → Storage, medicine_id → Medicine
Vaccine	vaccine_id, vaccine_name, target_disease, manufacturer, admin_route, recommended_age_days, dose_per_bird, price_per_dose, cold_chain_temp, supplier_id, status, created_at	vaccine_id	supplier_id → Supplier
Vaccine_Stock	storage_id, vaccine_id, doses_available, expiry_date, batch_number	(storage_id, vaccine_id)	storage_id → Storage, vaccine_id → Vaccine

GALALA UNIVERSITY
Powered by
Arizona State University®

Section 3: Farm Core

Entity	Attributes	PK	FK
Breed	breed_id, breed_name, origin, target_fcr, avg_weight_day42, target_mortality_pct, description, status, created_at	breed_id	—
Shed	shed_id, shed_name, capacity, area_sqm, status, created_at	shed_id	—
Cycle	cycle_id, breed_id, shed_id, initial_chicks, chick_price_per_unit, start_date, end_date, cycle_status, manager_id, created_at	cycle_id	breed_id → Breed, shed_id → Shed, manager_id → Manager
Assigned_To	cycle_id, person_id, assigned_date	(cycle_id, person_id)	cycle_id → Cycle, person_id → Farmer_Worker
Daily_Log *(weak)*	log_id, cycle_id, log_date, temp_celsius, humidity, ammonia_level, water_consumption, notes, created_by, created_at	(log_id, cycle_id)	cycle_id → Cycle, created_by → Worker
Daily_Feed *(weak)*	log_id, cycle_id, feed_type_id, quantity, price	(log_id, cycle_id, feed_type_id)	(log_id, cycle_id) → Daily_Log, feed_type_id → Feed_Type
Daily_Mortality *(weak)*	log_id, cycle_id, dead_count, cause	(log_id, cycle_id)	(log_id, cycle_id) → Daily_Log
Treatment	treatment_id, cycle_id, medicine_id, treatment_date, quantity_used, treatment_type, administered_by, created_at	treatment_id	cycle_id → Cycle, medicine_id → Medicine, administered_by → Farmer_Worker
Vaccination_Schedule	schedule_id, cycle_id, vaccine_id, scheduled_date, planned_birds, status, assigned_worker_id, created_at	schedule_id	cycle_id → Cycle, vaccine_id → Vaccine, assigned_worker_id → Worker
Vaccination_Record	record_id, schedule_id, administered_by, administered_at, birds_vaccinated, batch_number, cost, notes, created_at	record_id	schedule_id → Vaccination_Schedule, administered_by → Worker
Expense	expense_id, cycle_id, category, category_id, description, notes, amount, expense_date, created_by, created_at	expense_id	cycle_id → Cycle, category_id → Expense_Category, created_by → Worker
Expense_Category	category_id, name, description, parent_category_id, budget_limit, created_at	category_id	parent_category_id → Expense_Category (self-ref)
Quality_Control	qc_id, cycle_id, inspector_id, inspection_date, inspection_type, sample_size, test_results, compliance_status, corrective_action, created_at	qc_id	cycle_id → Cycle, inspector_id → Technician

Section 4: B2B Commercial Layer

Entity	Attributes	PK	FK
Customer	customer_id, company_name, trade_name, customer_type, phone, email, city, address, credit_limit, payment_terms, status, account_manager_id, created_at, updated_at	customer_id	account_manager_id → Manager
Contact_Person	contact_id, customer_id, full_name, job_title, phone, email, is_primary, created_at	contact_id	customer_id → Customer
Sales_Order	order_id, order_number, customer_id, cycle_id, order_date, delivery_date, birds_qty, est_weight_kg, price_per_kg, total_amount (VIRTUAL), status, payment_status, created_at	order_id	customer_id → Customer, cycle_id → Cycle
Sale	sale_id, cycle_id, manager_id, customer_id, order_id, sale_date, birds_sold, total_weight_kg, price_per_kg, total_revenue (VIRTUAL), created_at	sale_id	cycle_id → Cycle, manager_id → Manager, customer_id → Customer, order_id → Sales_Order
Invoice	invoice_id, invoice_number, sale_id, customer_id, issue_date, due_date, amount, tax_amount, status, created_at	invoice_id	sale_id → Sale, customer_id → Customer
Payment	payment_id, invoice_id, payment_date, amount, method, reference_number, notes, created_at	payment_id	invoice_id → Invoice

Section 5: Equipment & Maintenance

Entity	Attributes	PK	FK
Device_Type	type_id, type_name, description, created_at	type_id	—
Device	device_id, device_name, type_id, fuel_type, status, shed_id, storage_id, created_at	device_id	type_id → Device_Type, shed_id → Shed, storage_id → Storage
Device_Usage *(weak)*	usage_id, device_id, cycle_id, usage_date, hours, cost	(usage_id, device_id, cycle_id)	device_id → Device, cycle_id → Cycle
Maintains	maintenance_id, person_id, device_id, maintenance_datetime, description, status, cost, created_at	maintenance_id	person_id → Technician, device_id → Device

Section 6: HR & Training

Entity	Attributes	PK	FK
Attendance	attendance_id, person_id, work_date, check_in, check_out, status, notes, created_at	attendance_id	person_id → Worker
Payroll	payroll_id, person_id, payment_date, period_start, period_end, basic_salary, overtime_hours, overtime_amount, bonuses, deductions, net_amount, status, created_at	payroll_id	person_id → Worker
Performance_Review	review_id, person_id, reviewer_id, review_date, score, performance_score, attendance_score, quality_score, teamwork_score, overall_score (VIRTUAL), comments, next_review_date, created_at	review_id	person_id → Worker, reviewer_id → Manager
Training	training_id, title, description, trainer, scheduled_date, duration_hours, start_date, end_date, location, max_participants, cost, status, created_at	training_id	—
Training_Enrollment	enrollment_id, training_id, person_id, enrollment_date, status, score, certificate_issued, certificate_url, created_at	enrollment_id	training_id → Training, person_id → Worker

Section 7: Audit

Entity	Attributes	PK	FK
Audit_Log	audit_id, person_id, action, entity, details, entity_type, entity_id, old_values, new_values, ip_address, created_at	audit_id	person_id → Worker

GALALA UNIVERSITY
Powered by
Arizona State University®

4. Triggers and Business Rule Enforcement

The schema implements **16 database triggers** to enforce complex business rules that cannot be expressed through simple CHECK constraints:

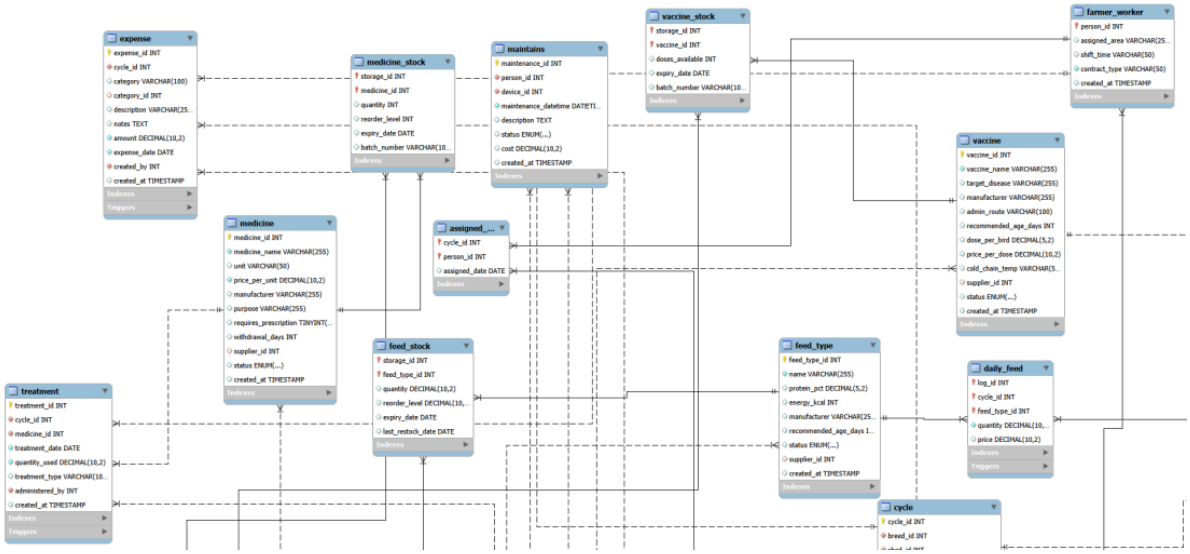
#	Trigger	Event	Purpose
T0	`before_insert_cycle`	BEFORE INSERT on Cycle	Enforces that only one active cycle can exist per shed at any time
T1	`before_insert_daily_log`	BEFORE INSERT on Daily_Log	Validates that the log date falls within the cycle's date range and the cycle is Active
T2	`before_insert_treatment`	BEFORE INSERT on Treatment	Validates treatment date within cycle period; rejects treatments on non-Active cycles
T3	`before_insert_vaccination_record`	BEFORE INSERT on Vaccination_Record	Prevents vaccination from being recorded before its scheduled date
T4	`after_insert_vaccination_record`	AFTER INSERT on Vaccination_Record	Automatically updates the corresponding Vaccination_Schedule status to 'Completed'
T5	`before_insert_performance_review`	BEFORE INSERT on Performance_Review	Validates that the review date is between the worker's hire date and the current date
T6	`before_insert_expense`	BEFORE INSERT on Expense	Validates expense date within cycle period; rejects expenses on non-Active cycles
T7	`trg_deduct_medicine_stock`	AFTER INSERT on Treatment	Automatically deducts medicine stock using FIFO (earliest expiry first); signals error if insufficient stock
T8	`trg_deduct_feed_stock`	AFTER INSERT on Daily_Feed	Automatically deducts feed stock using FIFO; signals error if insufficient stock
T9	`trg_deduct_vaccine_stock`	AFTER INSERT on Vaccination_Record	Automatically deducts vaccine doses using FIFO; signals error if insufficient doses
T10	`before_insert_sale`	BEFORE INSERT on Sale	Prevents selling more birds than are currently alive (initial − deaths − previous sales)
T11	`before_insert_mortality`	BEFORE INSERT on Daily_Mortality	Prevents recording mortality count exceeding the remaining living flock
T12	`after_insert_sale_autoclose`	AFTER INSERT on Sale	Automatically closes the cycle (sets status = 'Closed') when all birds have been sold
T13	`after_insert_mortality_autoclose`	AFTER INSERT on Daily_Mortality	Automatically closes the cycle when mortality wipes out the entire flock
T14	`before_insert_sale_order_check`	BEFORE INSERT on Sale	Validates that the linked Sales_Order belongs to the same cycle and same customer
T15	`after_insert_sale_update_order`	AFTER INSERT on Sale	Marks the linked Sales_Order as 'Delivered' and 'Paid' when a sale is recorded against it

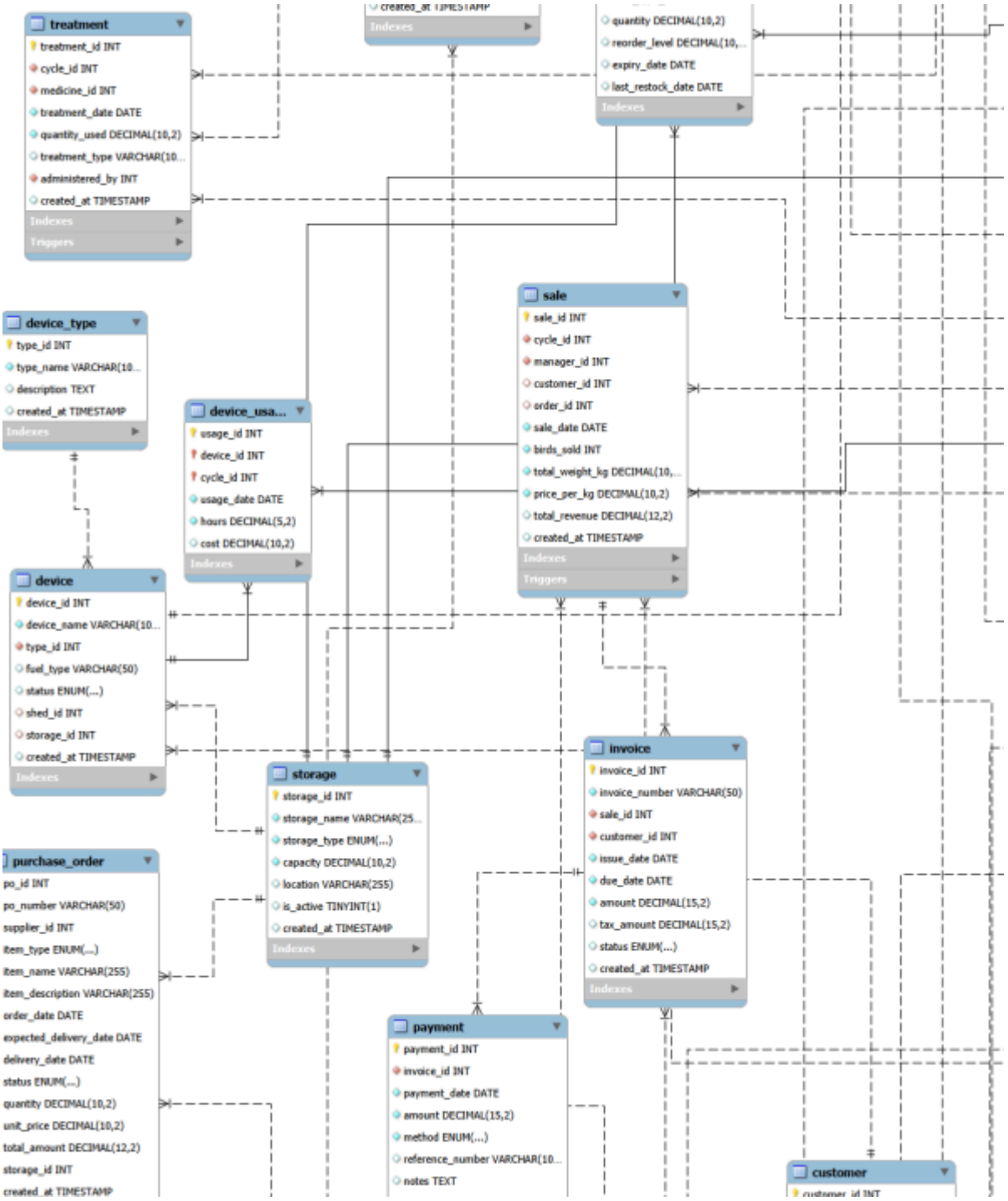
5. Database Views

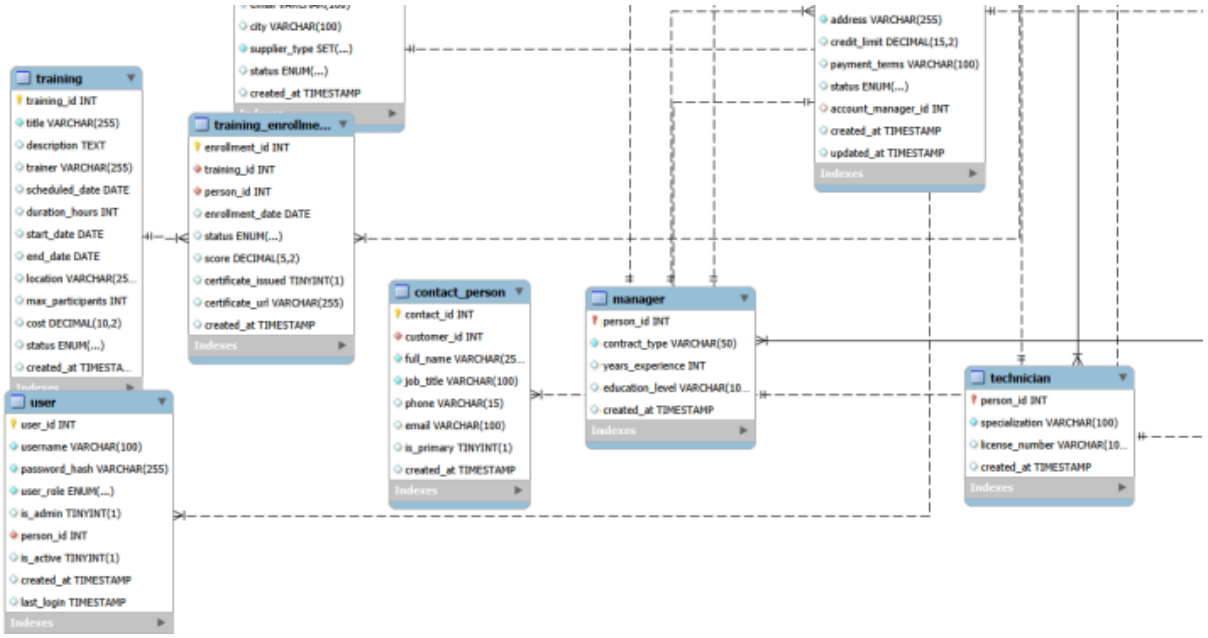
The schema defines **10 precomputed views** to optimize complex aggregate queries:

View	Purpose
`v_worker_details`	Joins Worker with Manager/Farmer_Worker/Technician subtables to show role-specific detail in a single query
`v_customer_financial_summary`	Aggregates total invoiced amount, total paid, and outstanding balance per customer
`v_cycle_status`	Real-time cycle dashboard: living birds, total mortality, mortality rate %, and days active
`Cycle_Living_Birds`	Computes living birds per cycle by subtracting both deaths and sold birds from initial count
`Cycle_Feed_Cost`	Aggregates total feed cost per cycle from Daily_Feed records
`Cycle_Profit`	Full P&L per cycle: total revenue − (chick cost + feed cost + treatment cost + other expenses)
`Inventory_Status`	Unified inventory status across feed, medicine, and vaccine stock with LOW/OUT/OK indicators
`Treatment_Details`	Joins Treatment with Medicine and Worker to show medicine name, administered-by name, and computed cost
`Worker_Attendance_Summary`	Aggregates attendance records per worker: total days, present, late, absent, and attendance rate %
`Worker_Payroll_Summary`	Aggregates payroll records per worker: total payments, total paid to date, last payment date, average bonus

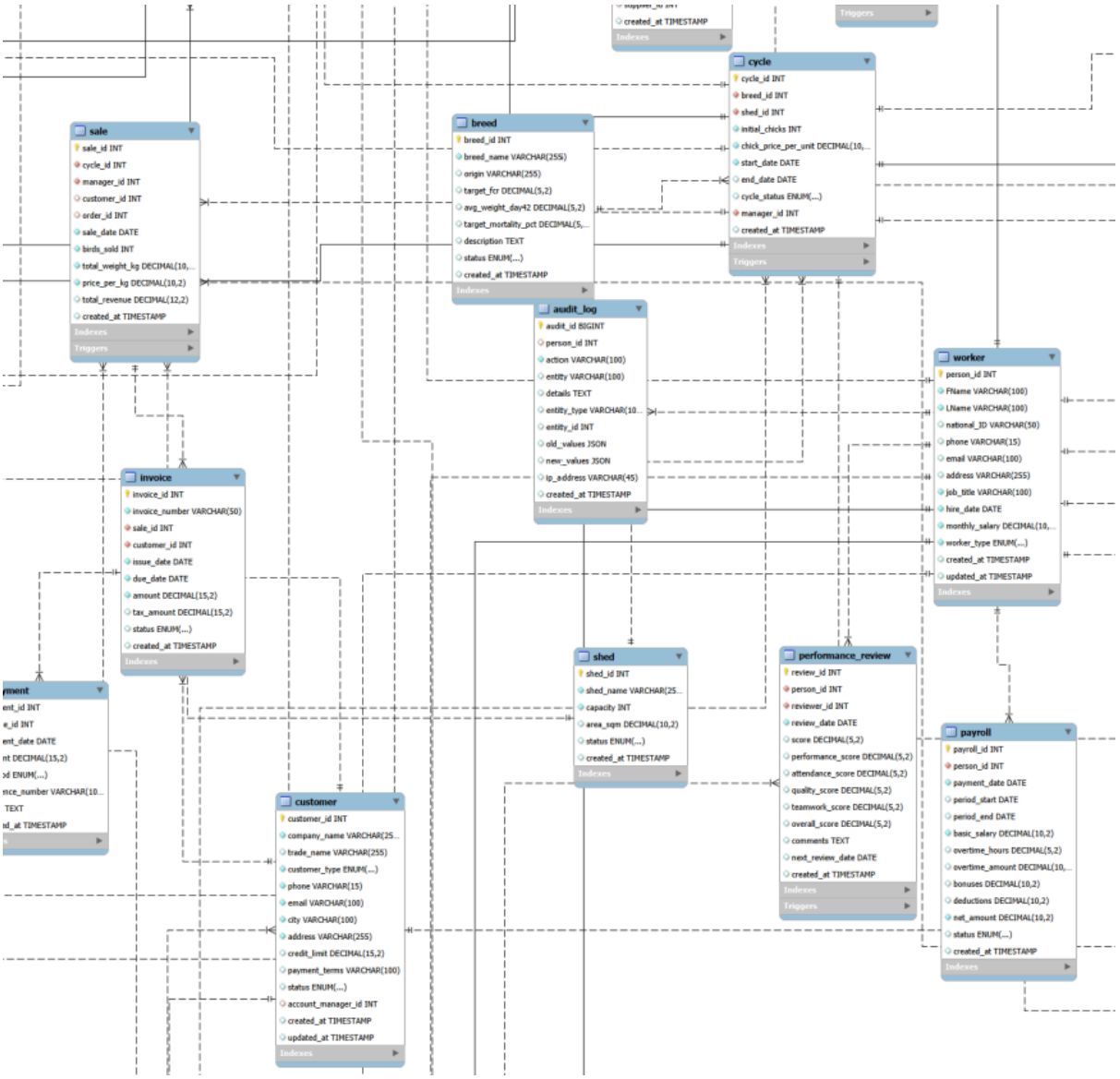
6. Relational Model

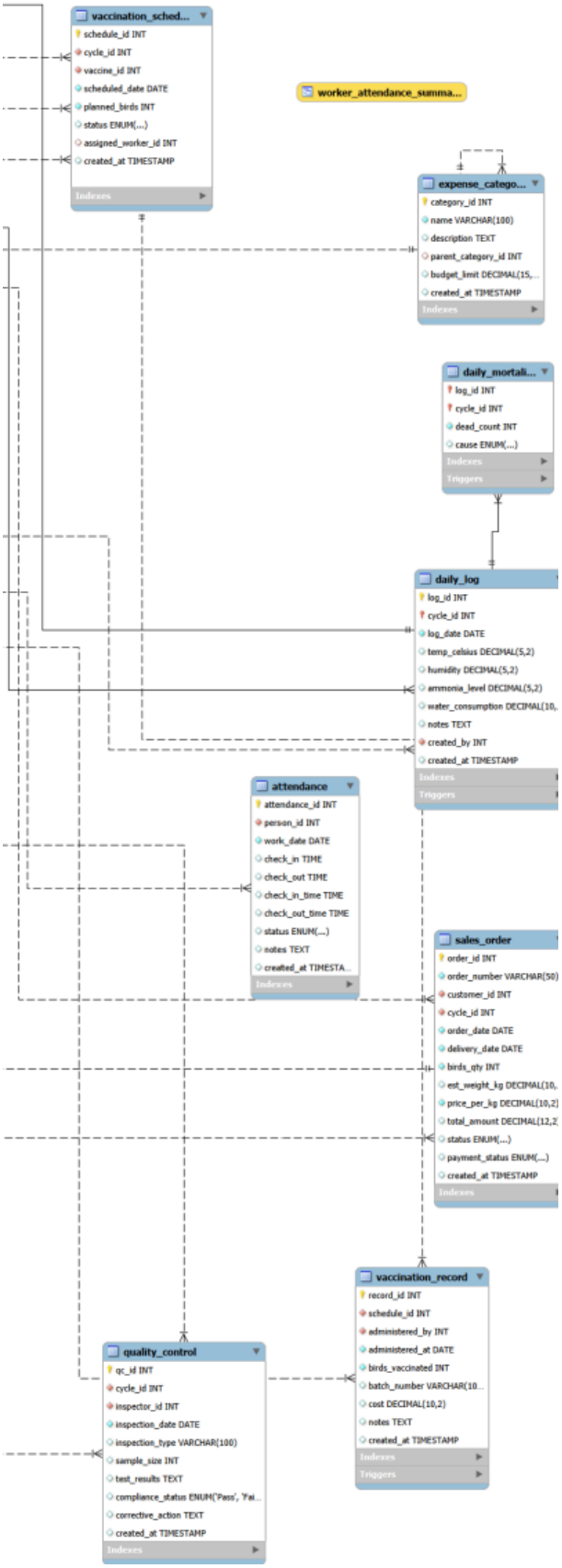



treatment
treatment_id INT
cycle_id INT
medicine_id INT
treatment_date DATE
quantity_used DECIMAL(10,2)
treatment_type VARCHAR(10...
administered_by INT
created_at TIMESTAMP
Indexes
Triggers
created_at TIMESTAMP
Indexes
quantity DECIMAL(10,2)
reorder_level DECIMAL(10,...
expiry_date DATE
last_restock_date DATE
Indexes
sale
sale_id INT
cycle_id INT
manager_id INT
customer_id INT
order_id INT
sale_date DATE
birds_sold INT
total_weight_kg DECIMAL(10,...
price_per_kg DECIMAL(10,2)
total_revenue DECIMAL(12,2)
created_at TIMESTAMP
Indexes
Triggers
device_type
type_id INT
type_name VARCHAR(10...
description TEXT
created_at TIMESTAMP
Indexes
device_usa...
usage_id INT
device_id INT
cycle_id INT
usage_date DATE
hours DECIMAL(5,2)
cost DECIMAL(10,2)
Indexes
device
device_id INT
device_name VARCHAR(10...
type_id INT
fuel_type VARCHAR(50)
status ENUM(...)
shed_id INT
storage_id INT
created_at TIMESTAMP
Indexes
storage
storage_id INT
storage_name VARCHAR(25...
storage_type ENUM(...)
capacity DECIMAL(10,2)
location VARCHAR(255)
is_active TINYINT(1)
created_at TIMESTAMP
Indexes
invoice
invoice_id INT
invoice_number VARCHAR(50)
sale_id INT
customer_id INT
issue_date DATE
due_date DATE
amount DECIMAL(15,2)
tax_amount DECIMAL(15,2)
status ENUM(...)
created_at TIMESTAMP
Indexes
purchase_order
po_id INT
po_number VARCHAR(50)
supplier_id INT
item_type ENUM(...)
item_name VARCHAR(255)
item_description VARCHAR(255)
order_date DATE
expected_delivery_date DATE
delivery_date DATE
status ENUM(...)
quantity DECIMAL(10,2)
unit_price DECIMAL(10,2)
total_amount DECIMAL(12,2)
storage_id INT
created_at TIMESTAMP
payment
payment_id INT
invoice_id INT
payment_date DATE
amount DECIMAL(15,2)
method ENUM(...)
reference_number VARCHAR(10...
notes TEXT
customer


training
training_id INT
title VARCHAR(255)
description TEXT
trainer VARCHAR(255)
scheduled_date DATE
duration_hours INT
start_date DATE
end_date DATE
location VARCHAR(25...
max_participants INT
cost DECIMAL(10,2)
status ENUM(...)
created_at TIMESTA...
training_enrollme...
enrollment_id INT
training_id INT
person_id INT
enrollment_date DATE
status ENUM(...)
score DECIMAL(5,2)
certificate_issued TINYINT(1)
certificate_url VARCHAR(255)
created_at TIMESTAMP
city VARCHAR(100)
supplier_type SET(...)
status ENUM(...)
created_at TIMESTAMP
address VARCHAR(255)
credit_limit DECIMAL(15,2)
payment_terms VARCHAR(100)
status ENUM(...)
account_manager_id INT
created_at TIMESTAMP
updated_at TIMESTAMP
contact_person
contact_id INT
customer_id INT
full_name VARCHAR(25...
job_title VARCHAR(100)
phone VARCHAR(15)
email VARCHAR(100)
is_primary TINYINT(1)
created_at TIMESTAMP
manager
person_id INT
contract_type VARCHAR(50)
years_experience INT
education_level VARCHAR(10...
created_at TIMESTAMP
technician
person_id INT
specialization VARCHAR(100)
license_number VARCHAR(10...
created_at TIMESTAMP
user
user_id INT
username VARCHAR(100)
password_hash VARCHAR(255)
user_role ENUM(...)
is_admin TINYINT(1)
person_id INT
is_active TINYINT(1)
created_at TIMESTAMP
last_login TIMESTAMP

GALALA UNIVERSITY
Powered by
Arizona State University®




vaccination_sched...
schedule_id INT
cycle_id INT
vaccine_id INT
scheduled_date DATE
planned_birds INT
status ENUM(...)
assigned_worker_id INT
created_at TIMESTAMP
Indexes
worker_attendance_summa...
expense_catego...
category_id INT
name VARCHAR(100)
description TEXT
parent_category_id INT
budget_limit DECIMAL(15,...
created_at TIMESTAMP
Indexes
daily_mortali...
log_id INT
cycle_id INT
dead_count INT
cause ENUM(...)
Indexes
Triggers
daily_log
log_id INT
cycle_id INT
log_date DATE
temp_celsius DECIMAL(5,2)
humidity DECIMAL(5,2)
ammonia_level DECIMAL(5,2)
water_consumption DECIMAL(10,...
notes TEXT
created_by INT
created_at TIMESTAMP
Indexes
Triggers
attendance
attendance_id INT
person_id INT
work_date DATE
check_in TIME
check_out TIME
check_in_time TIME
check_out_time TIME
status ENUM(...)
notes TEXT
created_at TIMESTA...
Indexes
sales_order
order_id INT
order_number VARCHAR(50)
customer_id INT
cycle_id INT
order_date DATE
delivery_date DATE
birds_qty INT
est_weight_kg DECIMAL(10,...
price_per_kg DECIMAL(10,2)
total_amount DECIMAL(12,2...
status ENUM(...)
payment_status ENUM(...)
created_at TIMESTAMP
Indexes
vaccination_record
record_id INT
schedule_id INT
administered_by INT
administered_at DATE
birds_vaccinated INT
batch_number VARCHAR(10...
cost DECIMAL(10,2)
notes TEXT
created_at TIMESTAMP
Indexes
Triggers
quality_control
qc_id INT
cycle_id INT
inspector_id INT
inspection_date DATE
inspection_type VARCHAR(100)
sample_size INT
test_results TEXT
compliance_status ENUM('Pass', 'Fai...
corrective_action TEXT
created_at TIMESTAMP
Indexes

GALALA UNIVERSITY
Powered by
Arizona State University®

Backend Database Implementation

1 Technology Stack

- **Database Engine:** MySQL (Version 8.0.45)
- **Runtime:** Node.js (Version v20+)
- **Web Framework:** Express.js (Version 5.2)
- **Template Engine:** EJS (Version 5.0)
- **Authentication:** bcryptjs + express-session (Version 3.0 / 1.19)
- **Security:** Helmet + express-rate-limit (Version 8.1 / 8.5)
- **AI Integration:** Google Generative AI (Gemini) (Version 0.24)
- **DB Driver:** mysql2/promise (Version 3.22)

2 Role-Based Access Control (RBAC)

The system enforces three-tier RBAC through dedicated middleware (middleware/rbac.js):

Role	Dashboard	Cycles	Logs	Operations	Supply Chain	Finance	HR	Reports	Management	QC	Training
Manager / Admin	✅ Full	✅ Create/Close	✅ Full	✅ Full	✅ Full	✅ Full	✅ Full	✅ Full	✅ Full	✅ Full	✅ Full
Farmer Worker	✅ Limited	✅ View Only	✅ Record	✅ Treatments Only	❌	❌	❌	❌	❌	✅ View	❌
Technician	✅ Limited	❌	❌	✅ Devices/Maintenance	❌	❌	❌	❌	❌	✅ Inspect	✅ View

security features:

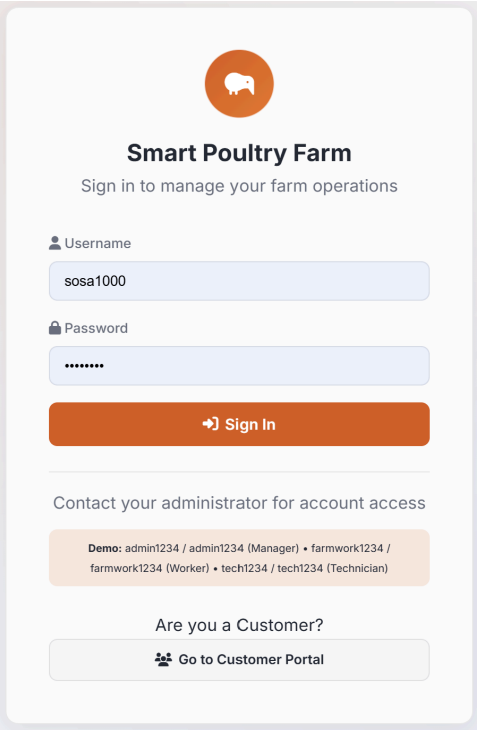
- **Password Hashing:** All passwords stored using bcryptjs with salt rounds
- **Session Management:** HTTP-only, strict SameSite cookies
- **Sensitive Field Filtering:** Salary, national ID, and password hashes are stripped from API responses for non-Manager roles
- **Cycle Access Control:** Farmer Workers can only access cycles they are assigned to
- **Audit Logging:** All significant actions are logged to the Audit_Log table with user ID, action type, and IP address

Frontend Design and Implementation

The client-side web application serves as a visual abstraction layer over the relational schema. Every interface element — buttons, forms, data tables, and stat cards — maps directly to structured SQL queries on the underlying tables and views.

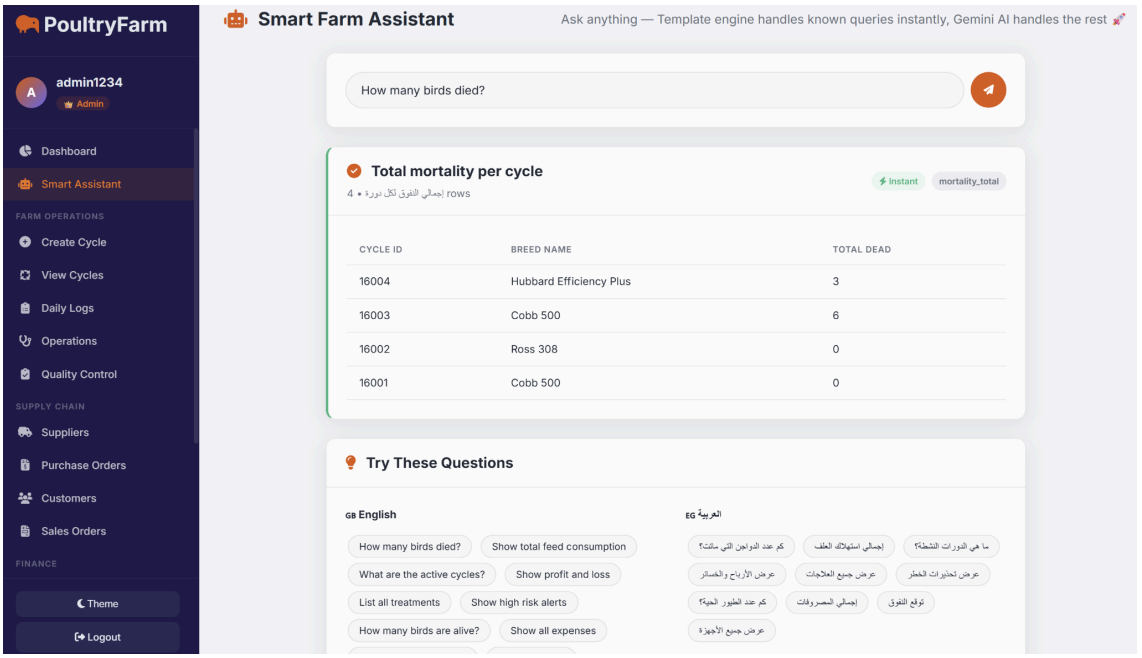
The interface uses the **Inter** font family, Font Awesome 6.4 icons, Chart.js for data visualization, and a responsive dark/light theme toggle. The sidebar navigation dynamically adapts based on the user's RBAC role.

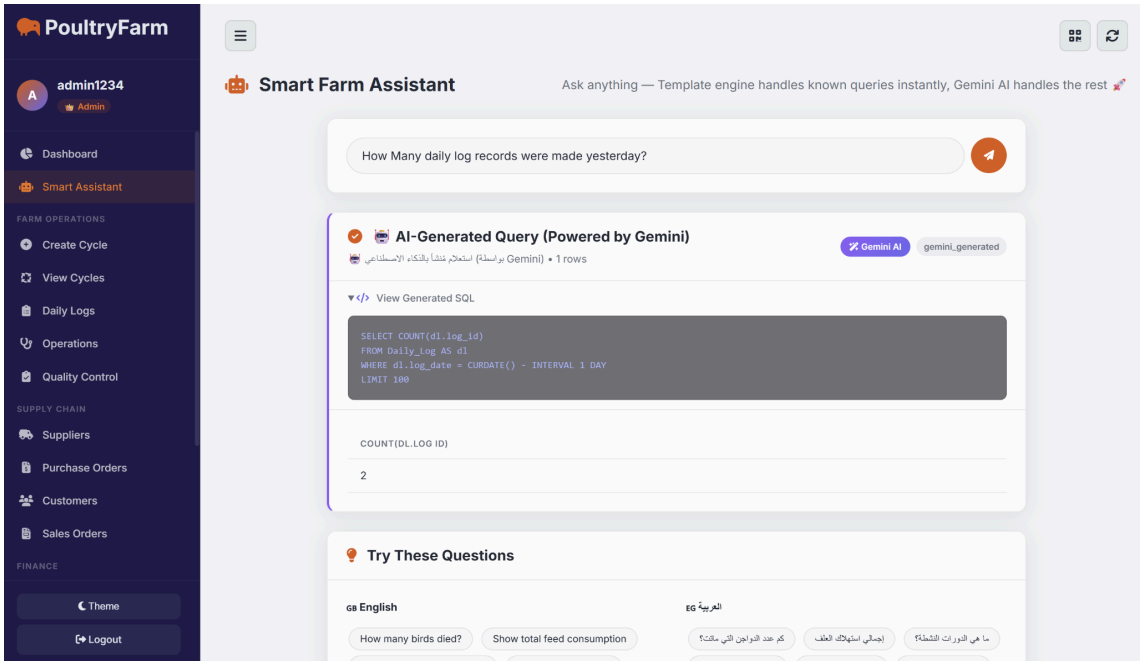
1 Login Page



The login page presents a clean authentication form with Username and Password fields. It binds to the User table, verifying credentials against the password_hash field using bcryptjs comparison. Upon successful login, the system creates a session containing user_id, person_id, user_role, and is_admin flag, then redirects to the Dashboard.

2 Smart AI Assistant



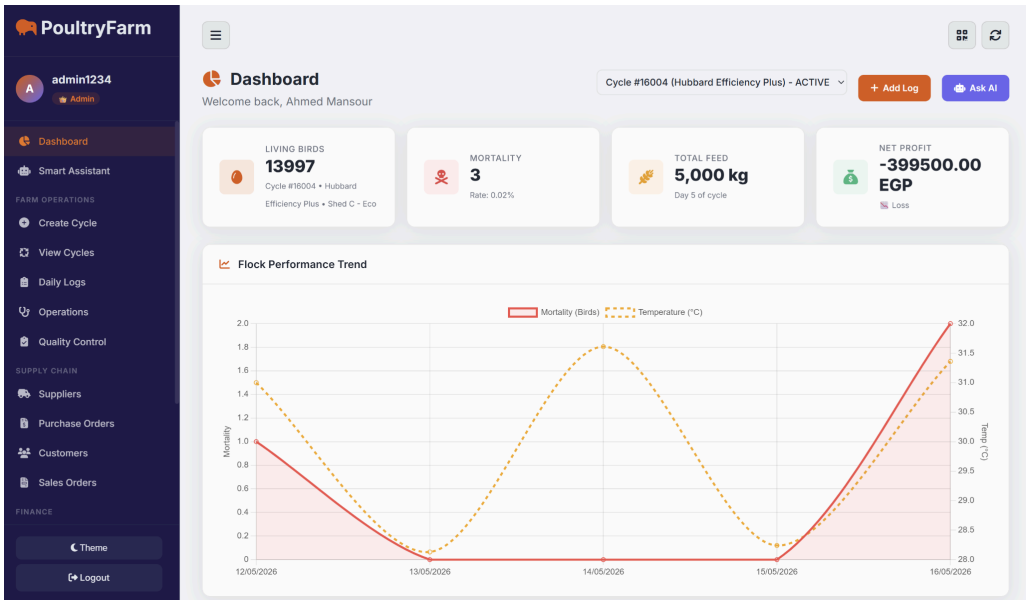


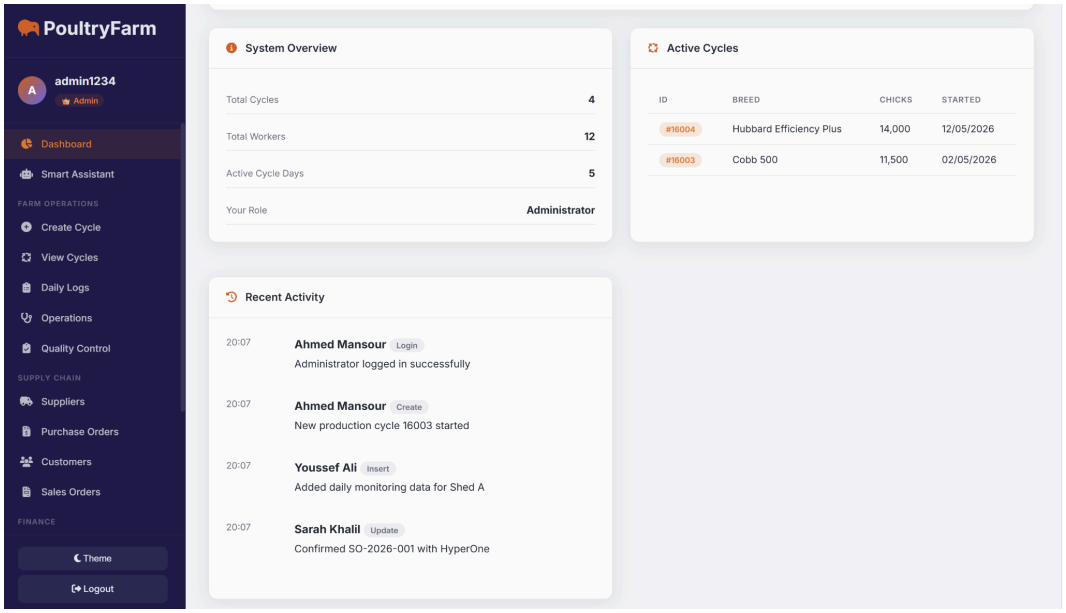
An integrated chat interface powered by Google Gemini AI. Managers can ask natural-language questions about farm performance, feed optimization, and production trends. The assistant has context about the database schema and can provide data-driven recommendations.

3 Dashboard

The main dashboard displays four KPI stat cards for the selected cycle:

- **Living Birds** — calculated from v_cycle_status view (initial chicks − deaths − sold)
- **Total Mortality** — sum of dead_count from Daily_Mortality
- **Total Feed** — sum of feed quantity from Daily_Feed
- **Net Profit** — from Cycle_Profit view (revenue − all costs)

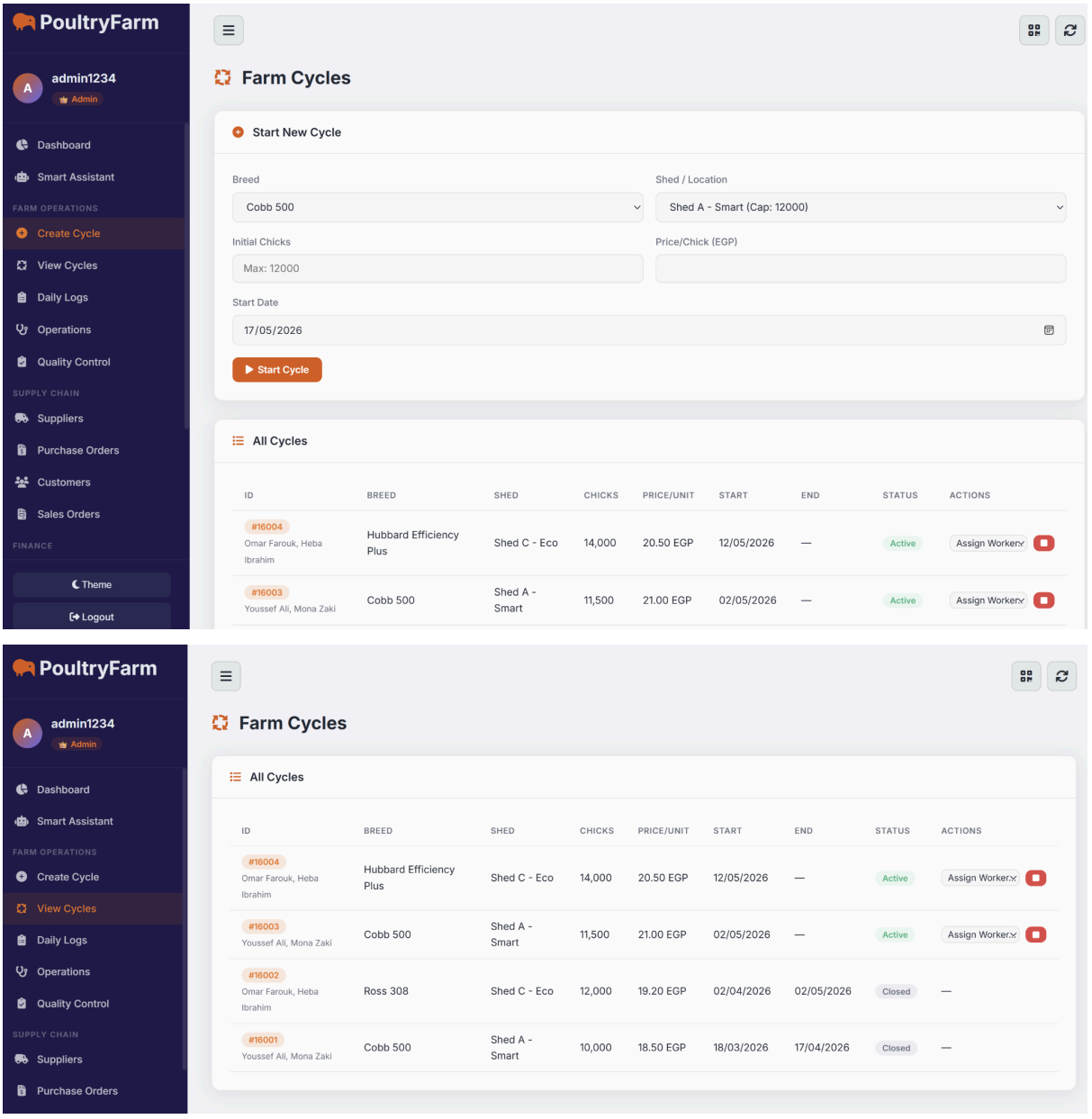




Below the stats, a **Flock Performance Trend** line chart (Chart.js) plots daily mortality count and temperature over time. The dashboard also shows a System Overview card (total cycles,

total workers), an Active Cycles table, and a Recent Activity timeline from the Audit_Log table. A cycle selector dropdown allows managers to switch between cycles.

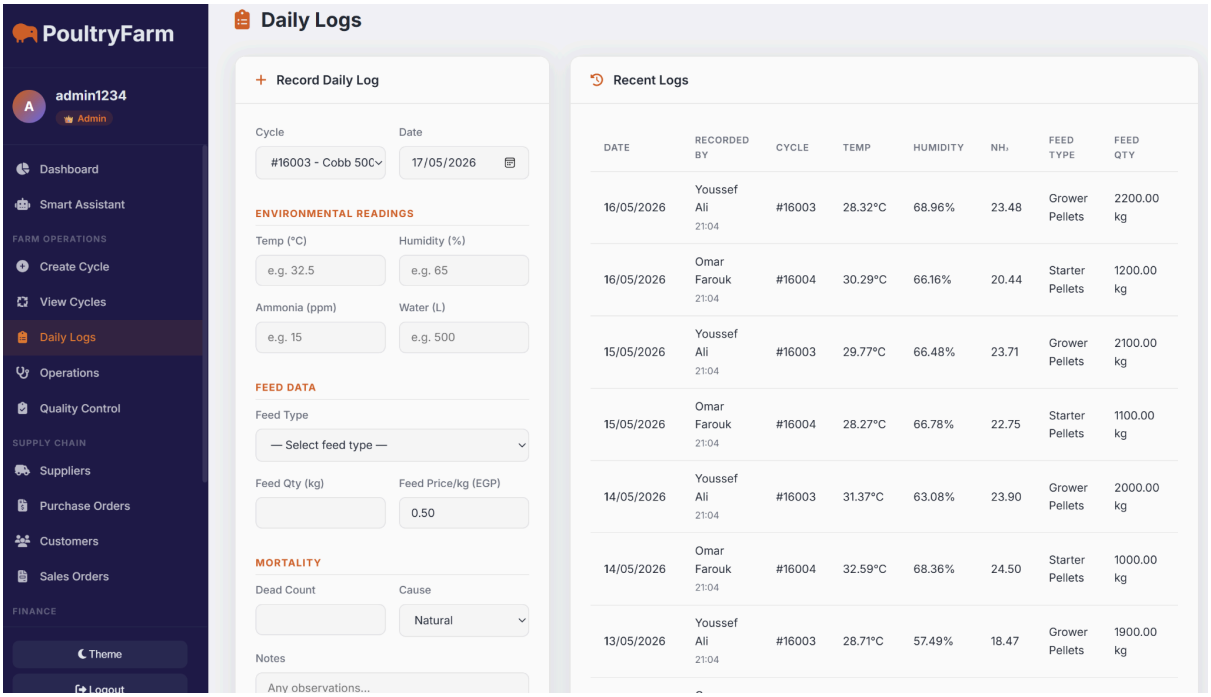
4 Cycles (Create & View)



The **Create Cycle** form collects Breed (dropdown from Breed), Shed (dropdown from Shed with capacity shown), Initial Chicks, Price per Chick, and Start Date. The shed capacity dynamically sets the max chick count. Submission inserts into the Cycle table.

The **All Cycles** table displays: ID, Breed, Shed, Chicks, Price/Unit, Start Date, End Date, Status, and Actions (Assign Worker dropdown, Close Cycle button). Workers can be assigned via the Assigned_To junction table directly from this interface.

5 Daily Logs

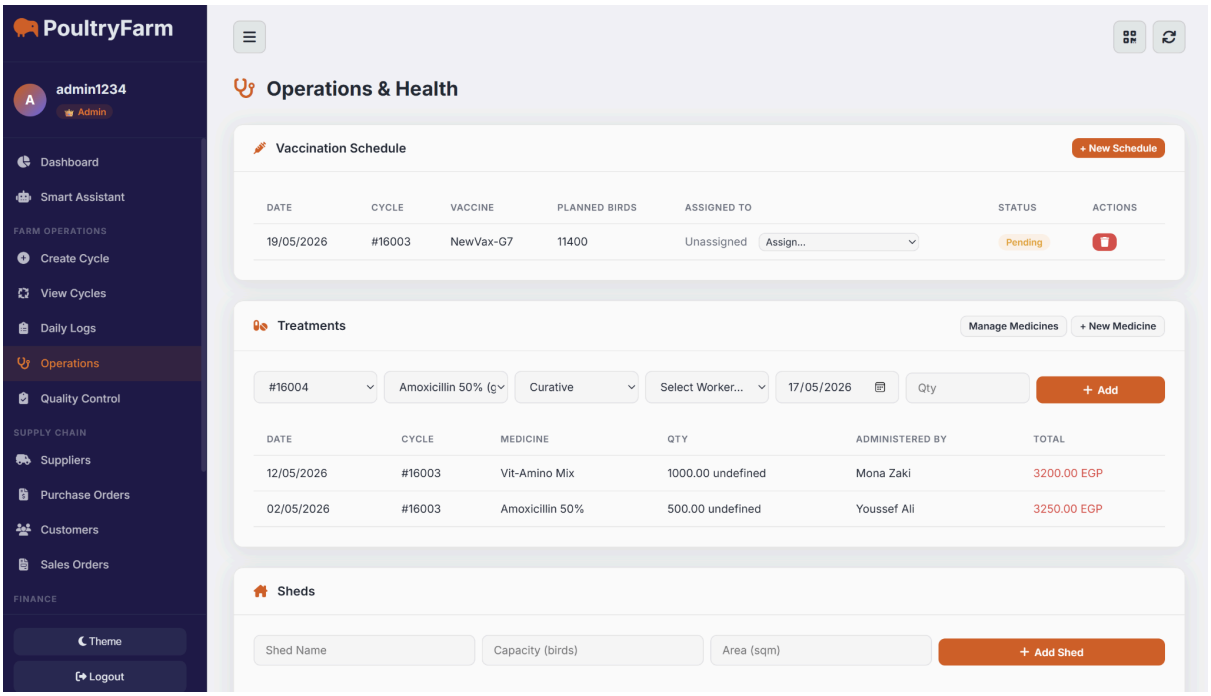


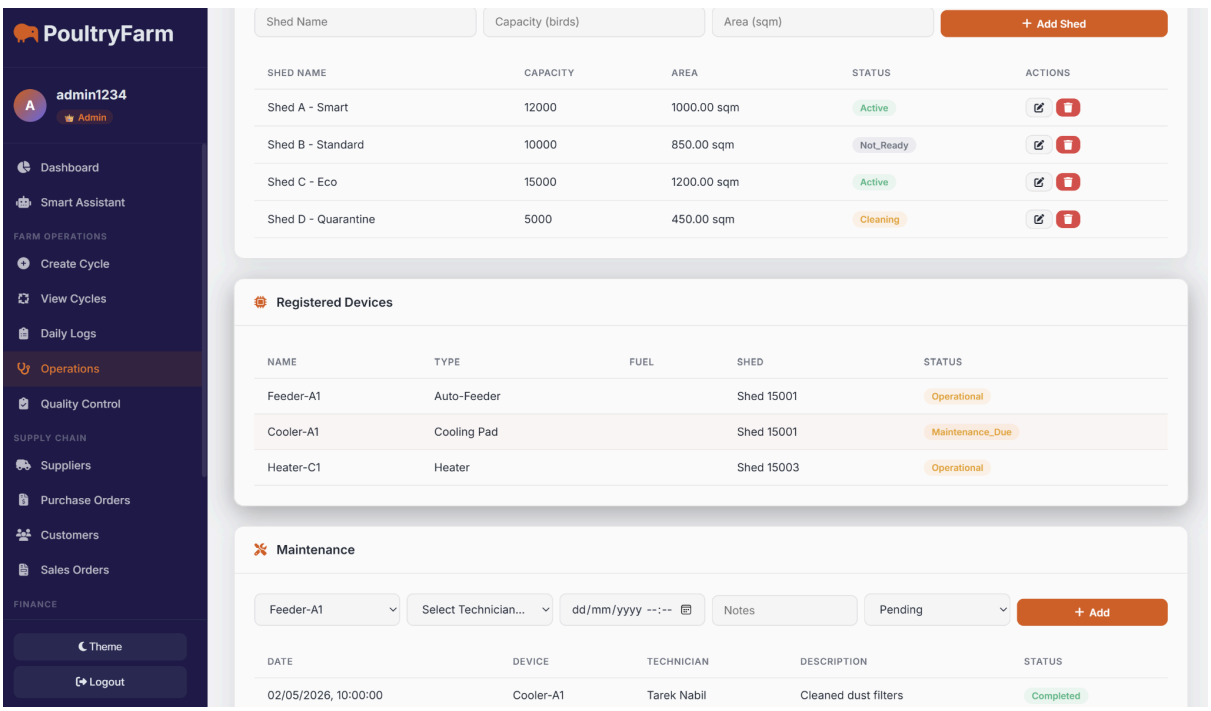
The log form is split into four sections:

1. **Cycle & Date** — cycle dropdown filtered to active/assigned cycles
2. **Environmental Readings** — Temperature (°C), Humidity (%), Ammonia (ppm), Water Consumption (L)
3. **Feed Data** — Feed Type dropdown (from Feed_Type), Quantity (kg), Price/kg
4. **Mortality** — Dead Count, Cause (Disease/Predator/Trampling/Heat_Stress/Unknown)

GALALA UNIVERSITY
Powered by
Arizona State University®

6 Operations & Health

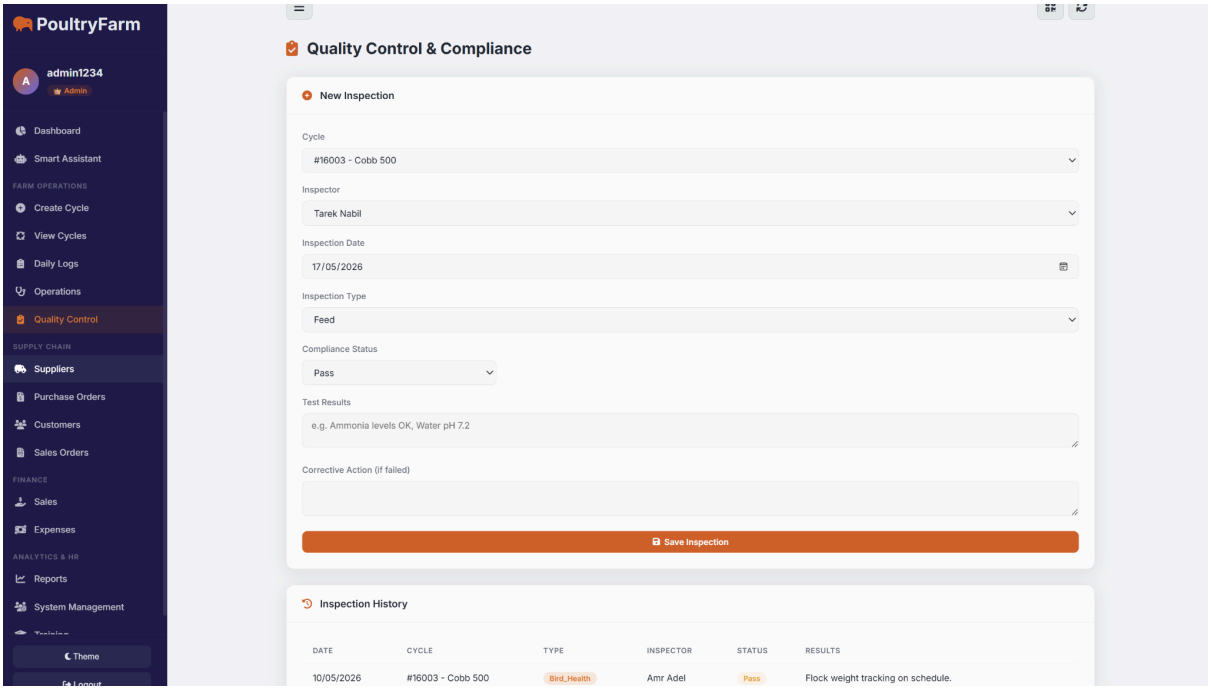




This page contains five sub-sections:

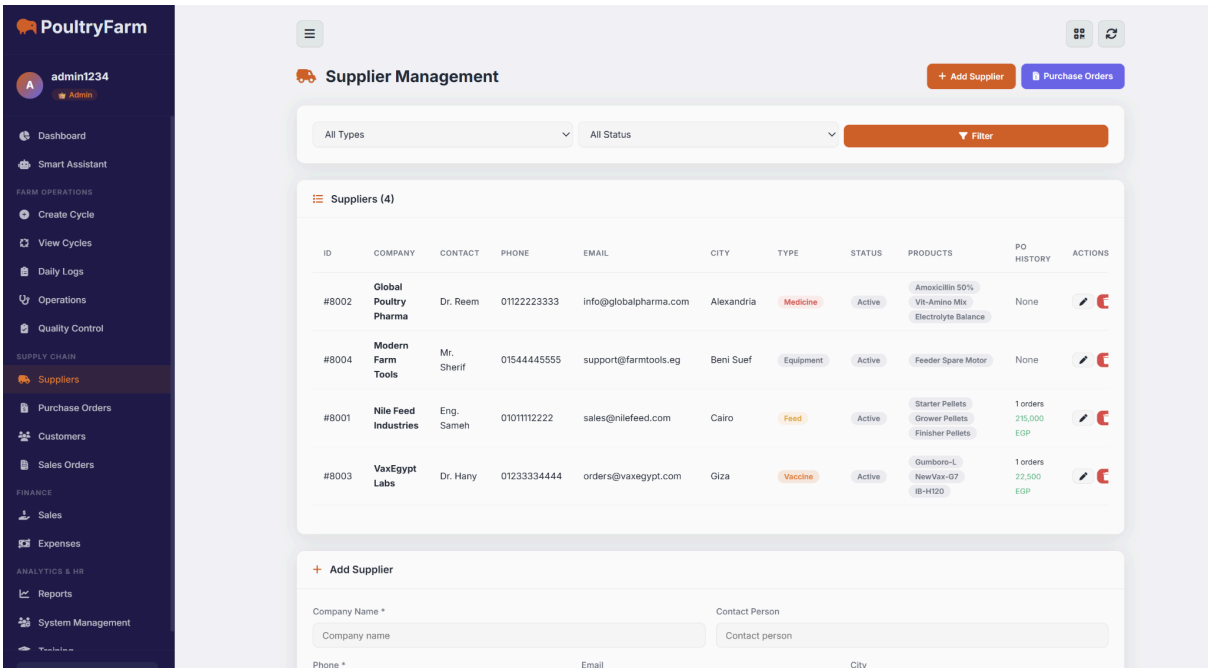
1. **Vaccination Schedule** — Table of all scheduled vaccinations with Date, Cycle, Vaccine, Planned Birds, Assigned Worker, and Status. Managers can create new schedules and assign workers.
2. **Treatments** — Form to record medicine treatments (Cycle, Medicine, Type, Administered By, Date, Quantity). The table shows treatment history with computed cost. A "Manage Medicines" modal allows adding/deleting medicine entries.
3. **Sheds** — Form to add new sheds and table listing all sheds with Name, Capacity, Area, Status. Edit modal allows updating shed details.
4. **Registered Devices** — Table of all devices with Name, Type, Fuel, Shed assignment, and Status. Data from Device joined with Device_Type.
5. **Maintenance** — Form and table for technician maintenance records on devices. Tracks Date, Device, Technician, Description, and Status.

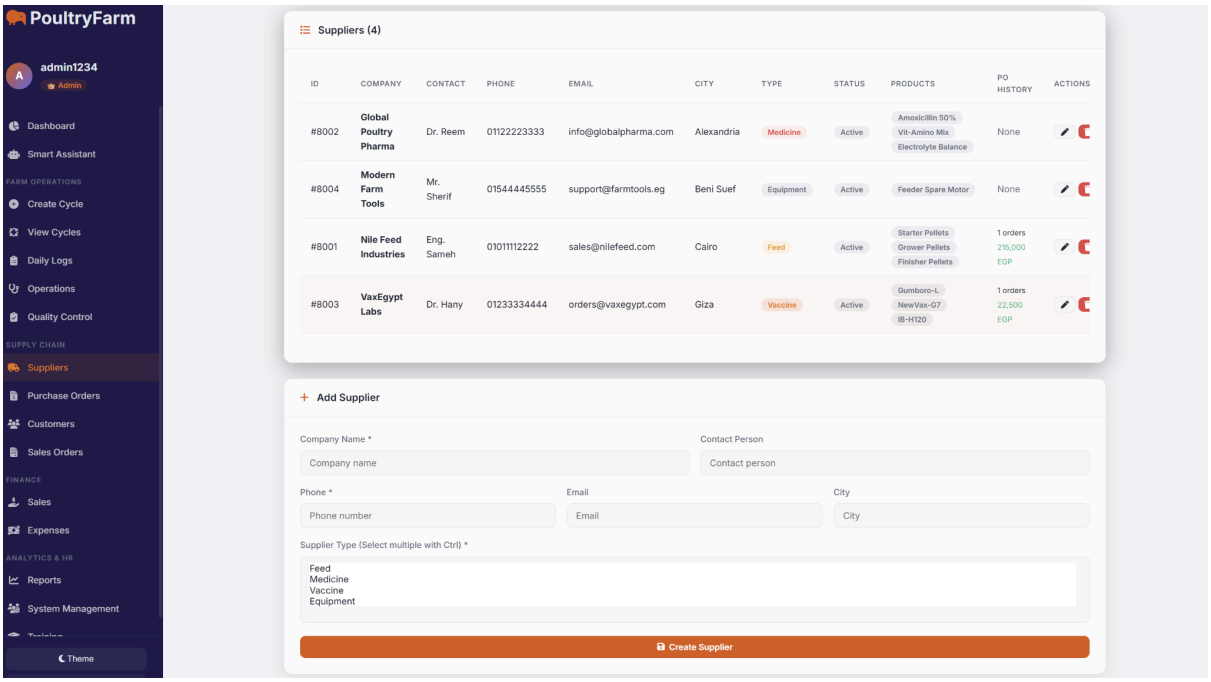
7 Quality Control



Displays QC inspection records from the Quality_Control table. Each record shows: Cycle, Inspector (Technician), Inspection Date, Type (Environment/Bird_Health), Compliance Status (Pass/Fail), Test Results, and Corrective Actions. Technicians can submit new inspections.

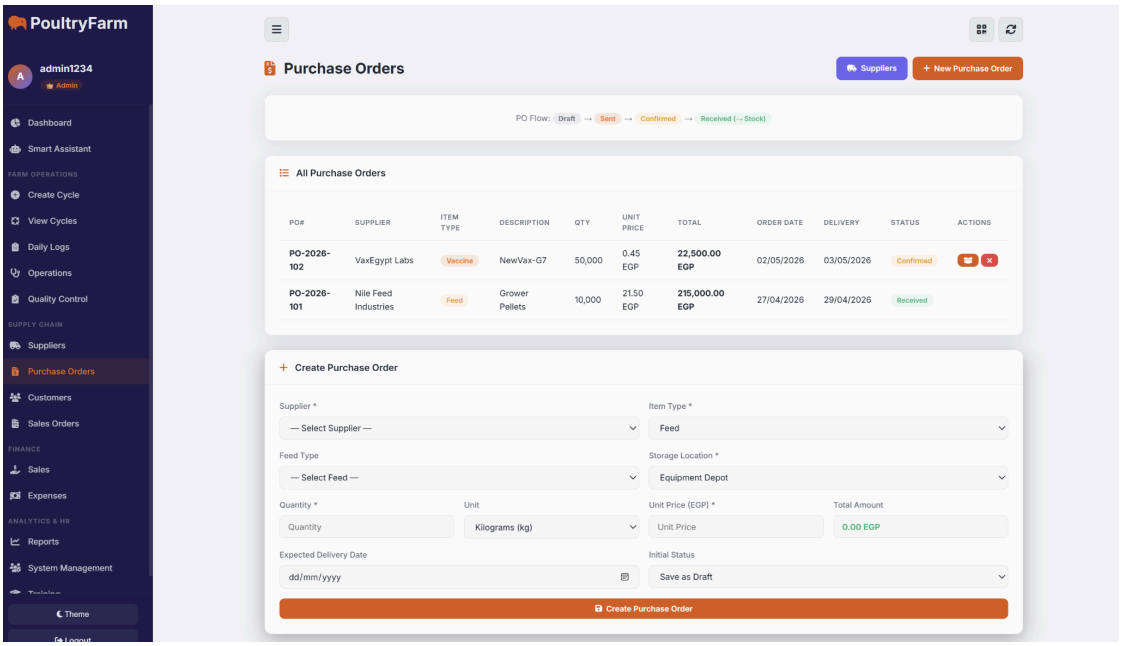
8 Suppliers





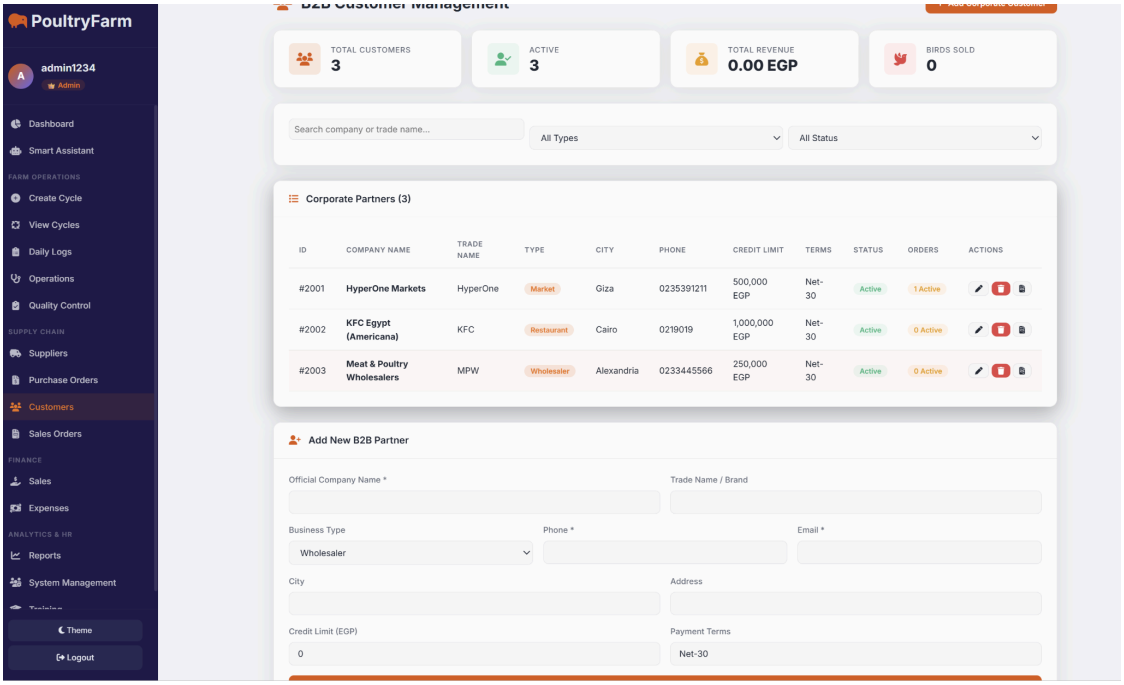
Manages external vendor profiles from the Supplier table. The form collects: Company Name, Contact Person, Phone, Email, City, and Supplier Type (multi-select: Feed/Medicine/Vaccine/Equipment). The table displays all suppliers with their status and type badges. Managers can edit and delete supplier entries.

9 Purchase Orders



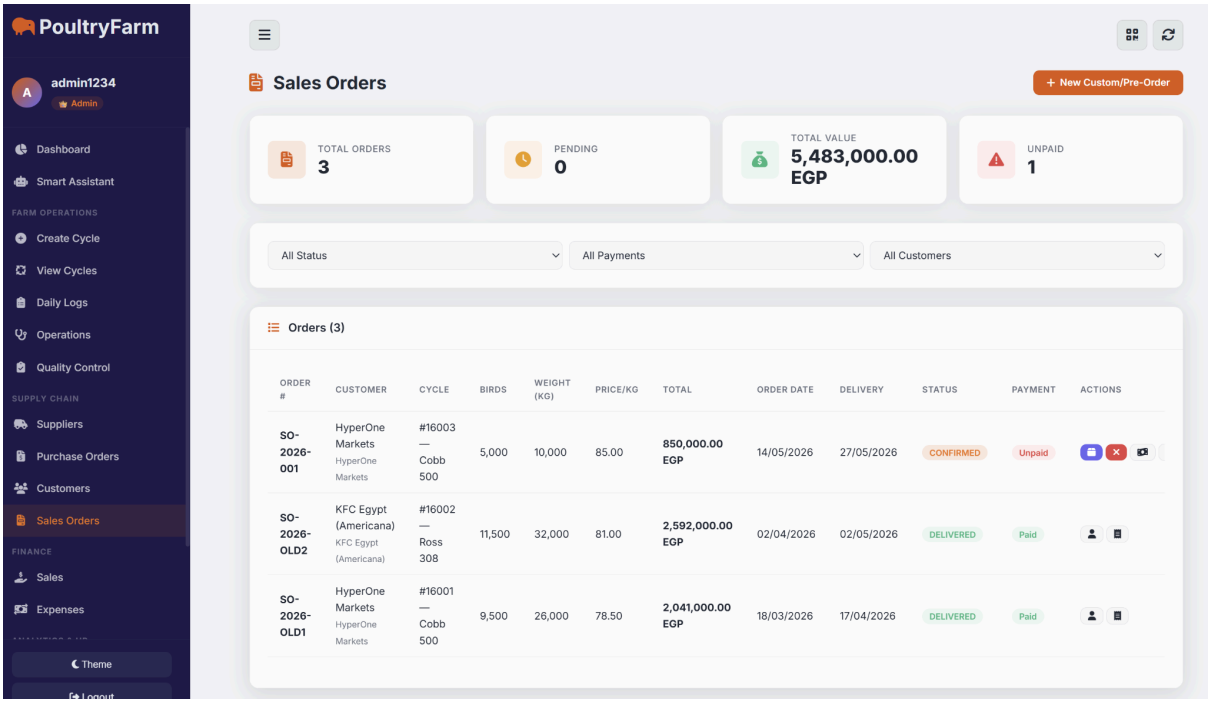
Manages procurement from the Purchase_Order table. The form collects: Supplier (dropdown), Item Type, Item Name, Description, Quantity, Unit Price, Order Date, Expected Delivery Date, and Storage Location. The total amount is auto-calculated (VIRTUAL column: quantity × unit_price). The table shows all POs with status badges (Draft/Sent/Confirmed/Received/Cancelled).

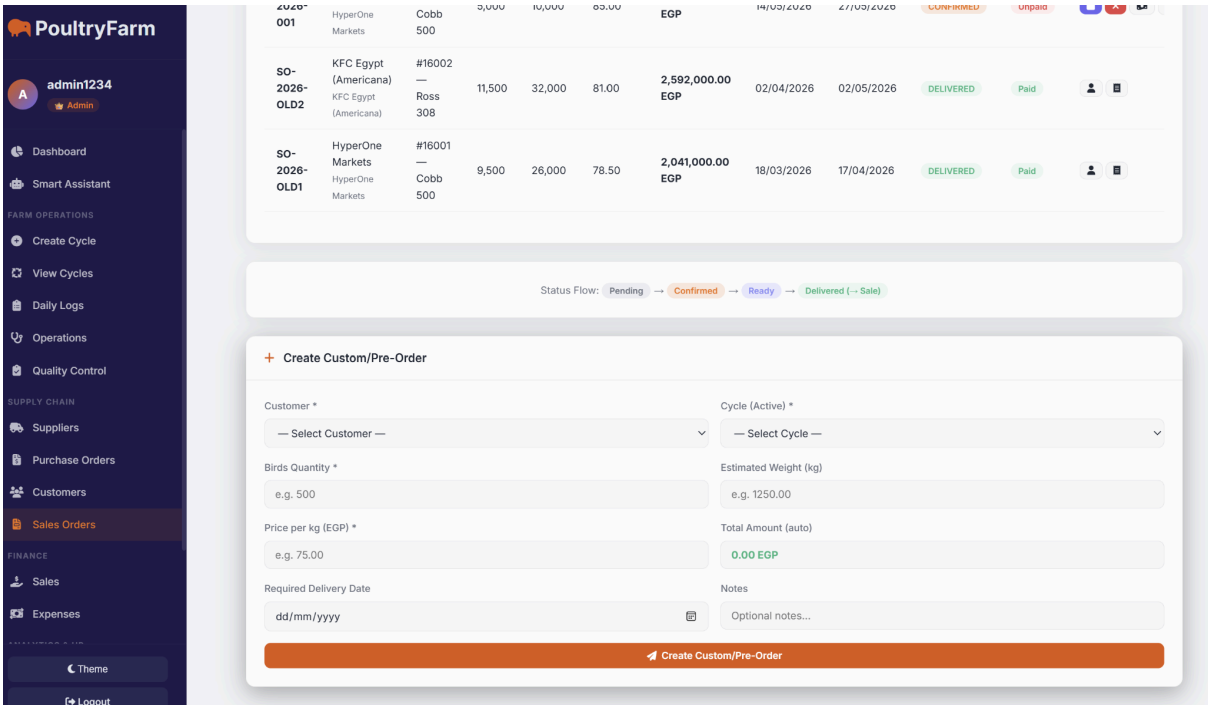
10 Customers



Manages B2B customer profiles from the Customer table. The form collects: Company Name, Trade Name, Customer Type (Wholesaler/Market/Restaurant/Distributor), Phone, Email, City, Address, Credit Limit, and Payment Terms. Each customer entry also supports Contact Persons (from Contact_Person table).

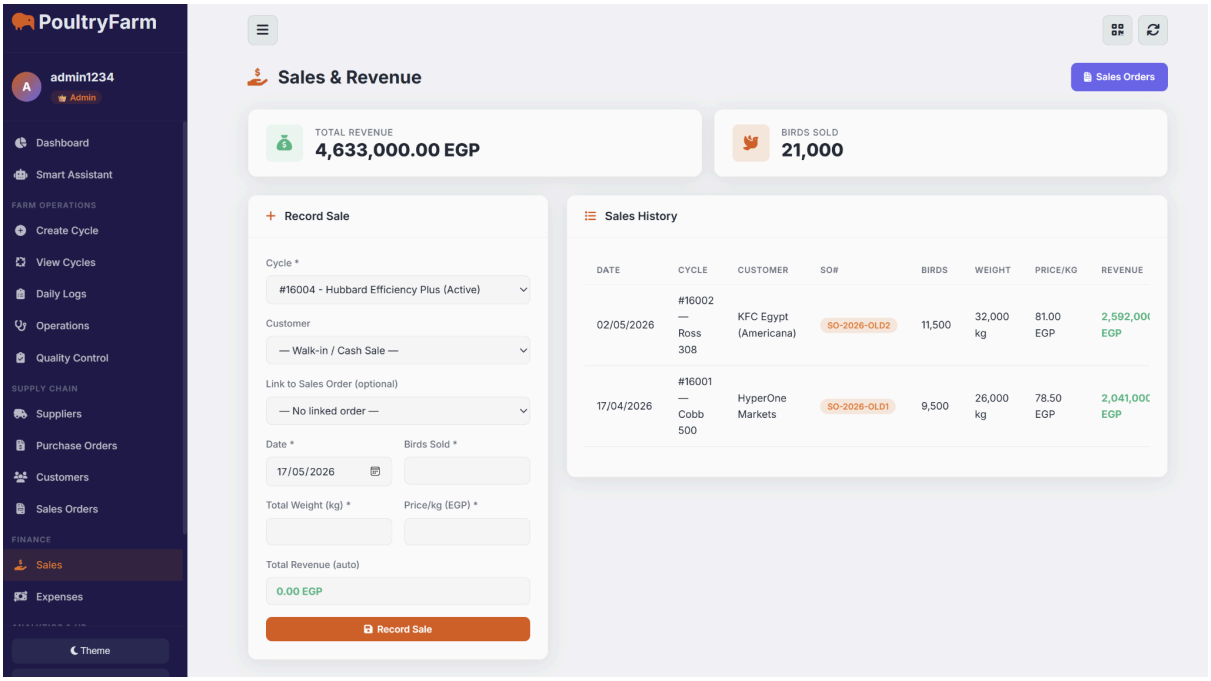
11 Sales Orders





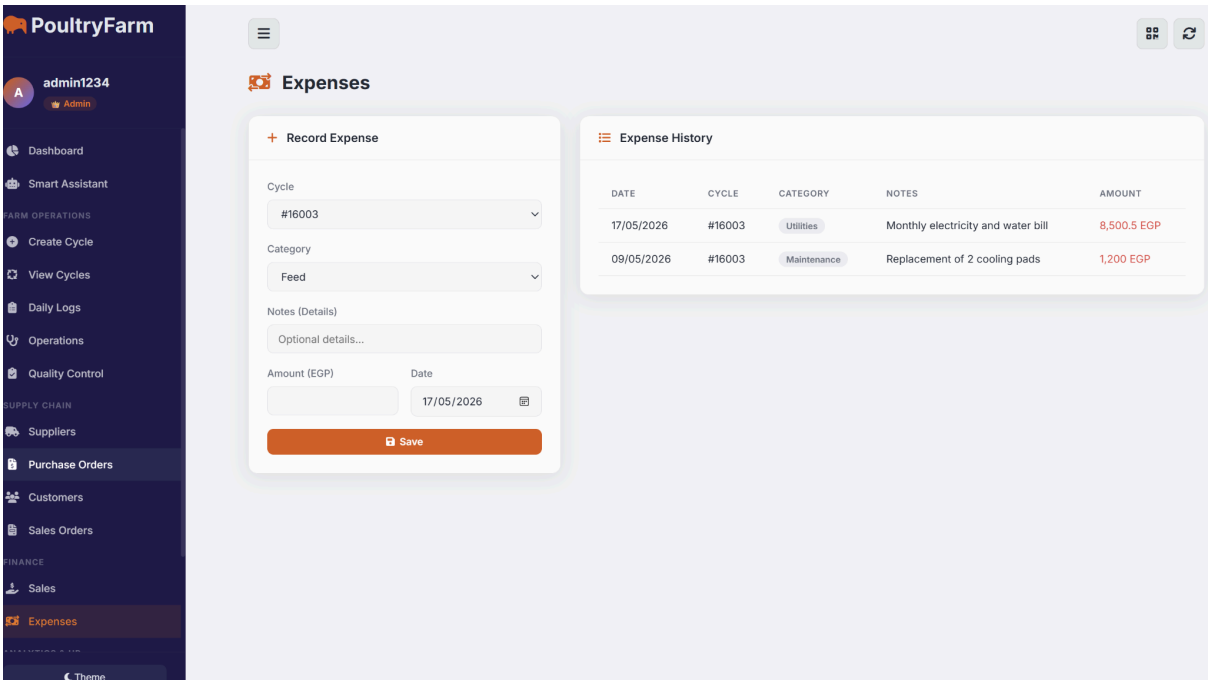
Manages pre-sale agreements from the Sales_Order table. The form collects: Customer, Cycle, Order Date, Delivery Date, Bird Quantity, Estimated Weight, and Price per kg. The total amount auto-calculates. The table shows order status workflow: Pending → Confirmed → Ready → Delivered. Orders are automatically marked "Delivered" when a linked Sale is recorded (via Trigger T15).

12 Sales & Revenue



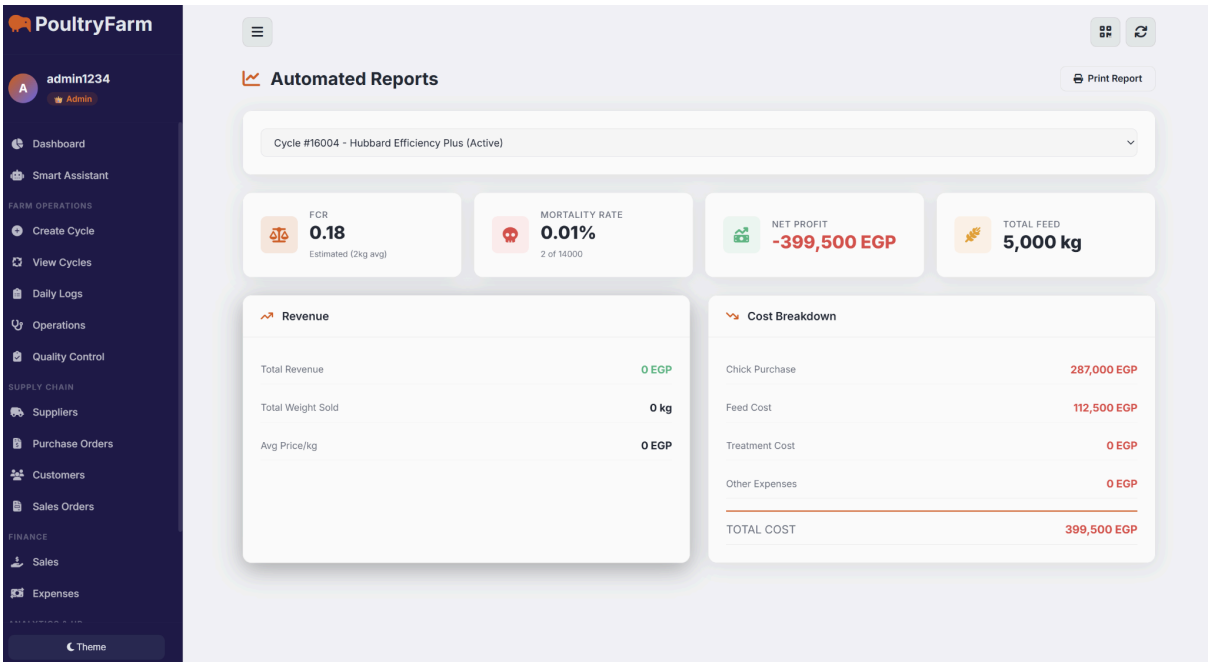
Displays two summary stat cards (Total Revenue, Total Birds Sold) computed via aggregate queries. The Record Sale form collects: Cycle, Customer (optional — supports walk-in sales), Linked Sales Order (optional), Date, Birds Sold, Total Weight, and Price/kg. A real-time revenue calculator shows the expected total. The Sales History table lists all past transactions.

13 Expenses



Records operational costs from the Expense table. The form collects: Cycle, Category, Amount, Date, and Notes. The table lists all expenses with cycle association. Expenses feed directly into the Cycle_Profit view for real-time P&L calculation.

14 Reports

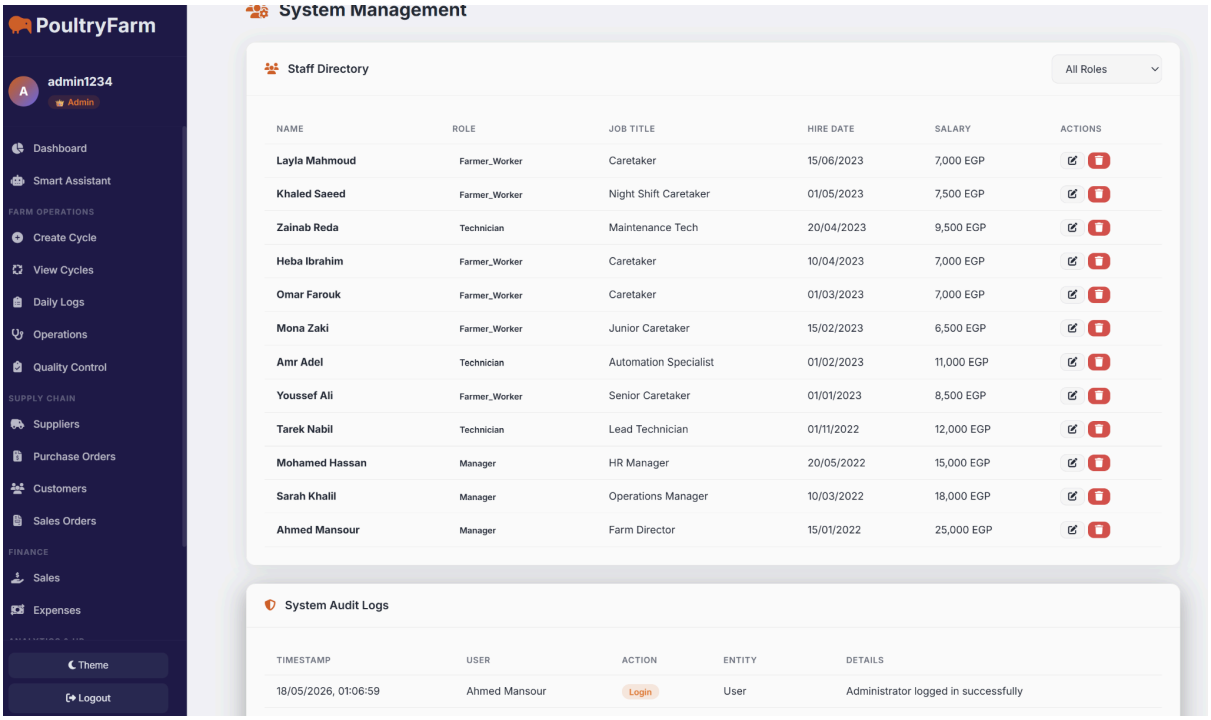


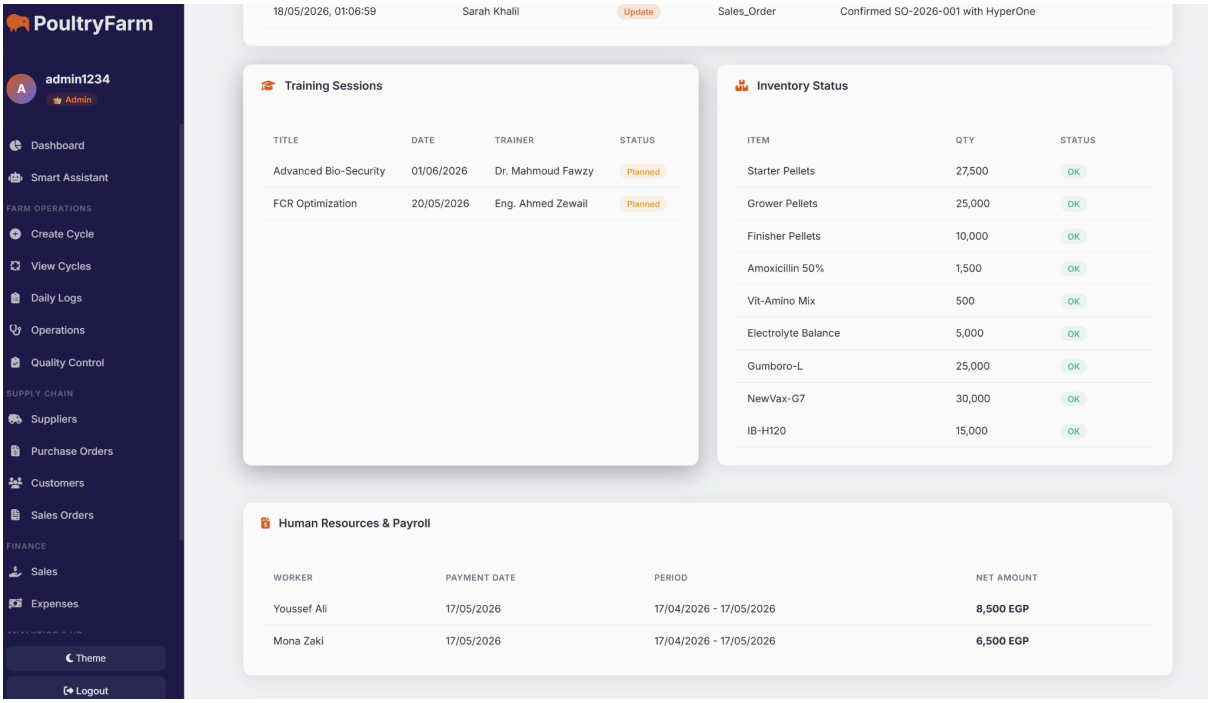
A per-cycle analytics dashboard. Users select a cycle from a dropdown, and the page displays:

- **FCR** (Feed Conversion Ratio) — computed from total feed ÷ total weight sold
- **Mortality Rate %** — total dead ÷ initial chicks × 100
- **Net Profit** — from Cycle_Profit view
- **Total Feed** consumed
- **Revenue Breakdown** — total revenue, total weight sold, average price/kg
- **Cost Breakdown** — chick purchase, feed cost, treatment cost, other expenses, total cost

Includes a **Print Report** button for hardcopy output.

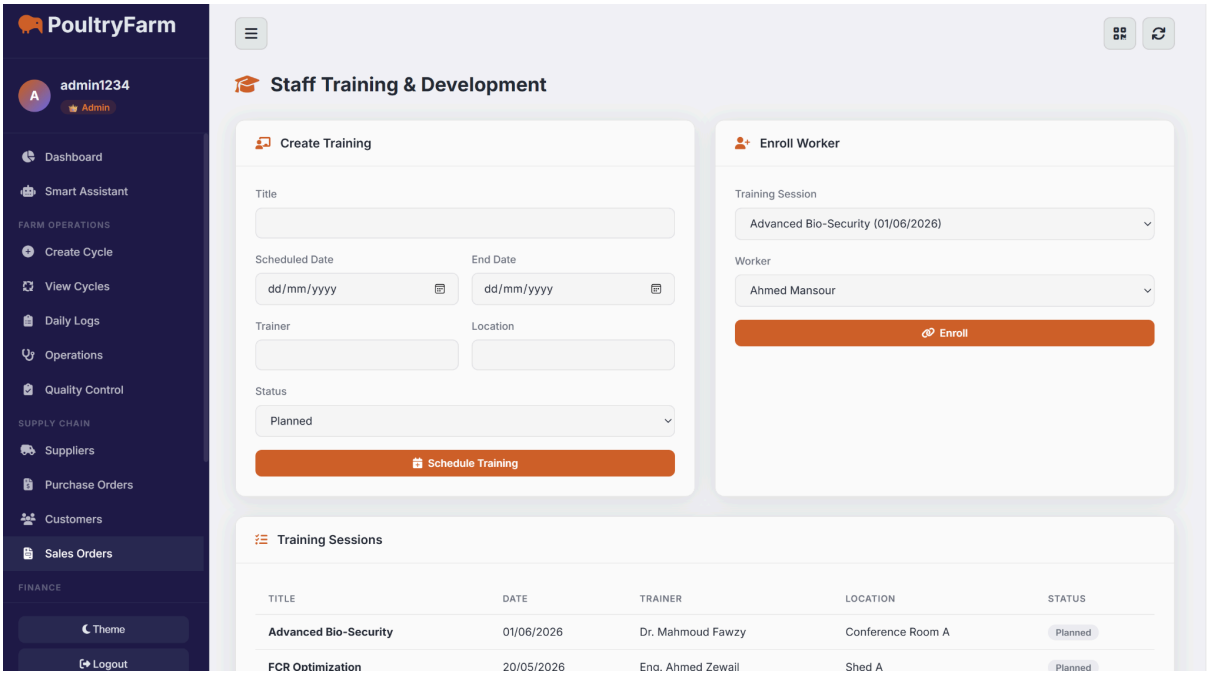
15 System Management

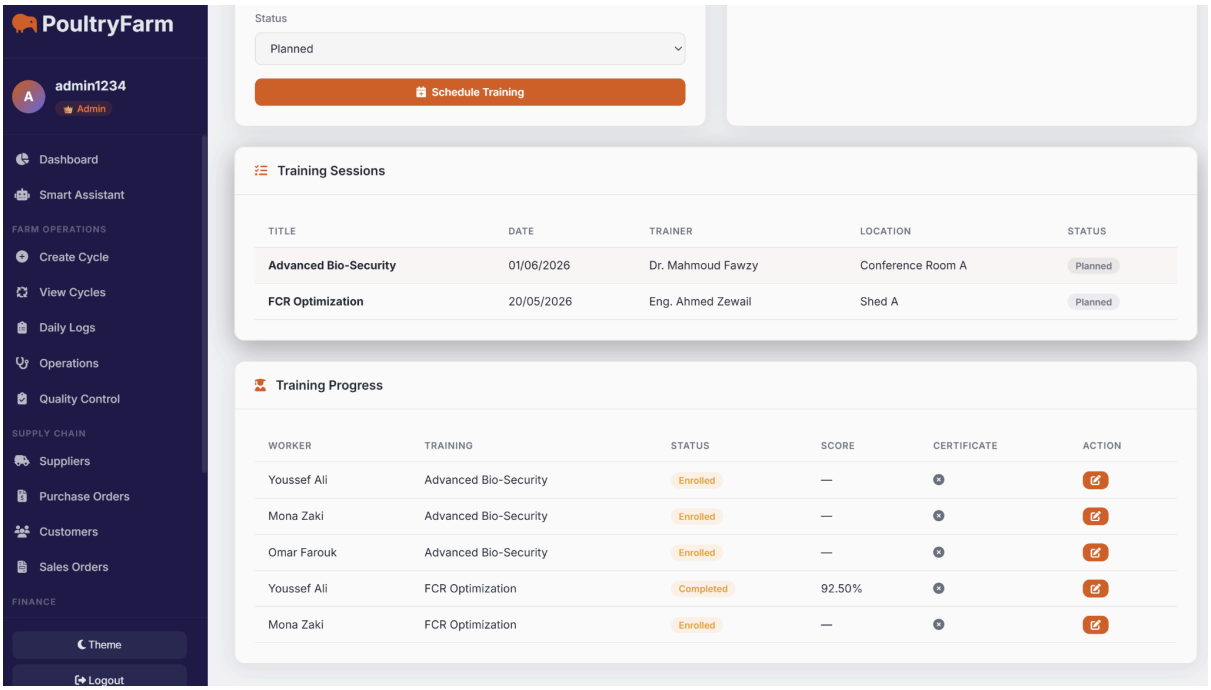




Admin-only interface for managing worker accounts, breeds, feed types, storage facilities, and system configuration. Provides CRUD operations on reference data tables.

16 Training





Manages training programs from the Training and Training_Enrollment tables. Shows scheduled sessions with Title, Trainer, Date, Duration, Location, Max Participants, and Status. Managers can create new training programs and enroll workers. Enrollment records track status, score, and certificate issuance.

Solved Challenges

1 Enforcing Referential Integrity and Preventing Orphan Rows

When modeling the relationship between production cycles and high-frequency telemetry, deleting a completed cycle initially triggered cascading foreign key failures from dependent daily logs, feed records, and mortality entries. This was resolved by implementing strategic cascading policies: ON DELETE CASCADE for dependent child records (Daily_Log, Daily_Feed, Daily_Mortality, Expense) and ON DELETE RESTRICT for critical reference entities (Breed, Shed, Medicine) to prevent accidental data loss.

2 Real-Time Aggregate KPI Computation

Ensuring that modifications to daily log tables instantly propagated to the Dashboard's KPI cards was challenging. This was resolved by creating precomputed database Views (v_cycle_status, Cycle_Profit, Cycle_Living_Birds) that use optimized SUM, COUNT, and COALESCE aggregations. Any new sale, death, or feed entry immediately reflects in the dashboard without requiring application-level recalculation.

3 Preventing Inventory Over-Consumption

Without proper guards, treatments could deplete medicine stock below zero, and feed entries could exceed available inventory. This was resolved through AFTER INSERT triggers (T7, T8, T9) that check available stock before deduction and raise SIGNAL SQLSTATE '45000' errors when stock is insufficient, using FIFO ordering by expiry date.

4 Biological Constraint Enforcement

Selling more birds than are alive, or recording mortality exceeding the remaining flock, would corrupt business metrics. Triggers T10 and T11 compute the current living count (initial − Σdeaths − Σsold) in real time and reject invalid entries. Triggers T12 and T13 automatically close cycles when the flock reaches zero.

Conclusion and Future Directions

The Smart Poultry Farm Management System successfully delivers a cohesive, fully functional, and highly normalized 3NF relational database bound to a polished enterprise web application. The platform provides farm managers with complete visibility over breeding cycles, daily environmental and biological telemetry, supply chain procurement, B2B commercial workflows, HR operations, equipment maintenance, quality control, and financial performance — all through a single, role-secured interface.

Future Scope

1. **Automated Alert Triggers:** Database triggers that fire visual dashboard alerts when mortality increases by more than 2% within a rolling 7-day window, or when inventory stock drops below reorder levels.
2. **Predictive Analytics:** Leveraging historical Daily_Log data to train machine learning models that predict final harvest weight and market value weeks in advance, based on early feed consumption and growth patterns.
3. **IoT Sensor Integration:** Direct API pipelines from physical sensors (temperature, humidity, ammonia, water flow) in the sheds, eliminating manual data entry and enabling real-time automated environmental monitoring.
4. **Mobile-Native Application:** A dedicated mobile app for Farmer Workers to log daily data, scan QR codes for vaccination batch tracking, and receive push notifications for pending tasks.
5. **Multi-Farm Support:** Extending the schema to support multiple farm locations under a single corporate entity, enabling cross-farm analytics and centralized inventory management.

References

- Elmasri, R. & Navathe, S.B. (2015). Fundamentals of Database Systems (7th ed.). Pearson.
- Silberschatz, A., Korth, H.F., & Sudarshan, S. (2019). Database System Concepts (7th ed.). McGraw-Hill.
- MySQL 8.0 Reference Manual. Oracle Corporation. https://dev.mysql.com/doc/refman/8.0/en/
- Express.js Documentation. https://expressjs.com/
- Chart.js Documentation. https://www.chartjs.org/docs/